



UNSW Faculty of Computer Science and Engineering
School of Software Engineering
COMP4953 Undergraduate Thesis C Report

Streamlining Content Authoring and Immersion through Refactoring of the *Crescendo* VR Platform

Author: Aidan Tan (z5360925)

Thesis required for the degree of: Bachelor of Engineering (Honours) (Software)

Submission: 12th Aug 2026

Supervisors: Tim Dodds, Graham Hannah

Abstract

Virtual Reality (VR) offers an immersive medium for classical music experiences, but early prototype versions suffer from high content-authoring effort and performance bottlenecks on standalone headsets. This thesis details the architectural refactoring and optimisation of *Crescendo*, a cross-platform VR platform developed for the UNSW Music Performance Unit (MPU) to showcase UNSW's orchestral and choral ensembles.

Initial versions relied on a multi-camera, multi-stream pipeline that required manual 3D geometry mapping, creating a slow and tedious workflow for adding new recordings. To resolve these issues, the system was shifted to a simplified single-stream media pipeline with adherence to Data-Oriented Programming principles. By storing experience metadata, audio files, and video textures inside Unity Scriptable Object assets, rapid onboarding of new repertoire can be achieved without modifying scene hierarchies or source code. To balance authoring efficiency with user immersion, the virtual environment was redesigned into a cohesive indoor concert theatre featuring real-time, audio-reactive visual effects (FFT spectrum analysis, particle systems, and Shader Graphs) that match performance dynamics.

Testing on Meta Quest 3 headsets guided targeted optimisations to performance and design. Although hardware limits, such as WebGL audio spectrum data constraints and mobile GPU limits, prevented reaching a full 72 FPS across all platforms, the refactored architecture successfully cut onboarding effort, improved user retention, immersion and engagement, and enhanced platform adaptability and flexibility. This work delivers an accessible, performant, and reusable software framework for MPU ensembles, laying a strong technical foundation for potential future UNSW-wide use.

Acknowledgements

This project was supported by the UNSW Immersive Team, who develop educational VR apps for UNSW, and the Music Performance Unit, who manage concerts and music ensembles in UNSW. Gratitude is extended to the 10+ people involved in the 4th Aug 2026 (informal) preliminary testing session, and to all the 80+ musicians and AV professionals involved in the recording of three pieces in 2023 for exclusive use in this project. Without any of these teams and people, and their generous support and feedback, this project would not have been possible.

1. Introduction

Virtual Reality (VR) provides an immersive medium to deliver classical music and concert performances in novel ways [1]. While attending a live performance in a concert hall remains the ideal acoustic and emotional baseline [2][3], advances in audio design, presence, and spatial immersion allow virtual environments to replicate this experience with astonishing realism in accessible settings [4][5].

UNSW's exclusive Orchestra & Choir VR application, *Crescendo*, provides an immersive experience showcasing the UNSW Orchestra and Collegium Musicum Choir. Developed in 2023 by the UNSW Immersive Team for the Music Performance Unit (MPU), it combines ensemble recordings with artistic visualisers designed to complement the integrity of the music and its performers. Its objectives are as follows:

- **Immersive Content:** Expanding on the visual, aural and haptic features of the project while keeping the UNSW Orchestra and Choir performers at the centre of the experience.
- **Project flexibility:** Developing the template (i.e. content authoring) features to improve the project's reusability and adaptability to more music recordings
- **Impact and reach:** Adding and polishing features to make the project more accessible to a variety of UNSW groups, and readily deployable to users across different platforms
- **Marketing and engagement:** Enhancing user interactions which increase the project's visual appeal during events and advertising of the UNSW Media & Immersive Program

Crescendo was designed as a solution to the following problem statement:

How might we strategically transform the existing visual, audio and interactive features of the Crescendo project for flexibility, reusability and accessibility so that it can reach a wider target audience within UNSW with better engagement and retention?

This statement balances technical feasibility with broad impact by focusing on reusable content authoring, engaging interactive visuals to increase user retention, and accessibility options to support diverse platforms and users prone to motion sickness.



Fig. 1. Initial interface prototype with selection menu cubes.

Prior to this thesis commencing in October 2025, the alpha prototype featured an interactive menu using Extended Reality (XR) and Web toolkits, allowing users to select songs, themes, baton effects, and colour palettes via interactable menu cubes (see Fig. 1). Upon entering the visualisation, users were surrounded by giant rising 3D cubes projecting video recordings of the ensemble on each face (see Fig. 2). Ray casting allowed users to pause

individual cubes to view videos up close, concluding with a scrollable credits screen listing all participating performers.

However, this initial prototype presented severe content-authoring bottlenecks. Relying on a complex multi-camera and multi-microphone setup, developers had to manually map multiple video streams onto 3D geometry and synchronise disparate audio stems, creating a high-friction workflow that hindered rapid onboarding of new repertoire.

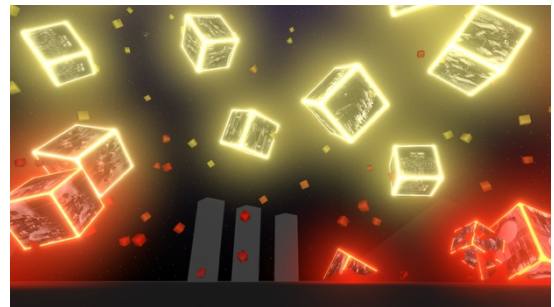


Fig. 2. Crescendo experience with coloured floating cubes that glow with Bloom.

A stakeholder review in January 2026 revised the project scope to prioritise authoring efficiency, accessibility, and maintainability. The architectural baseline pivoted to a single-video, single-audio stream architecture, drastically reducing authoring time while shifting the visual design focus to dynamic, audio-reactive spatial elements surrounding a central display.

Consequently, this thesis focuses on delivering a performant, accessible VR experience for MPU ensembles (which includes other ensembles not currently in the project such as the Corde Chamber Choir and the UNSW Wind Symphony), establishing a reusable template for broader university deployment.

The in-scope boundaries and deliverables of this thesis are defined as follows:

- A simplified media pipeline supporting one primary video stream and one unified audio source per performance.
- A standardised system for rapid content authoring, by integrating and syncing new recordings into Unity without manual geometry mapping.

- An audio-reactive environment (e.g., frequency-driven particle systems and spatial visual cues) surrounding a main display, enhancing presence without heavy post-processing.
- Optimisation targeting Standalone VR/MR Headsets and Web-based (WebGL/Desktop) platforms for cross-platform compatibility.
- An intuitive gaze/crosshair-based interaction model with visual comfort optimisations to mitigate motion sickness and accommodation-vergence strain.

The following, while still relevant for future work, are out of scope boundaries and deliverables for now as a result of the stakeholder meeting:

- Synchronised multi-perspective video arrays.
- Live multi-track spatial audio for each instrument section (replaced by single-source audio frequency analysis (FFT)).
- Broader templates for non-musical societies (relegated to the long-term product roadmap; many aspects of the project were still developed with this in mind).

This preliminary report will perform a literature review on appropriate topics related to the project and its applications in Section 2. An overview of the experimental nature of the project and its architectural stack can be found in Section 3. Section 4 then narrates the implementation process, including design changes performed, challenges encountered, optimisations made and trade-offs noted over the course of the development cycle. Section 6 synthesises both usability testing and performance testing results and the process of testing for each. Section 7 performs a reflection on the project and discusses its limitations and initiatives for future work. This is followed by a conclusion in Section 8.

2. Literature Review

The purpose of this literature review is to perform a comparative analysis of both academic research and practical applications across a range of key disciplines related to the *Crescendo* project. By synthesising theoretical frameworks across commercial and open-source benchmarks, this section aims to validate core technical concepts and foundational design principles, and identifies knowledge gaps critical to project implementation.

2.1.1. Classical Music in VR

Integrating virtual (or even mixed) reality technologies into classical music performance provides compelling opportunities for immersive concert design, as benchmarked by high-profile experiences like *Symphony VR* [7]. Survey research by Onderdijk et al. demonstrates that virtual concertgoers prioritise spatial presence, novelty, and proximity to performers [6]—qualities central to *Symphony VR*, which positions users directly within the LA Philharmonic. However, both Onderdijk et al. and *Symphony VR* reveal that extreme perceptual immersion could



Fig. 3. Gustavo Dudamel conducting in *Symphony VR*. Adapted from [7]

compromise the social and ritualistic dimensions of live classical performances, emphasising the need to balance individual immersion with communal coherence [6].

Achieving acoustic realism in orchestral VR relies heavily on spatial and haptic audio integration.

Vibrotactile and haptic studies demonstrate that audio-tactile synthesis deepens emotional resonance and spatial presence [8], complementing *Symphony VR*'s use of

position-accurate, multichannel audio recording to simulate acoustic seating within an ensemble. From an interaction perspective, metaverse concert research identifies usability, environmental fidelity, avatar synchronisation, and user autonomy as core experiential pillars [9]. While *Symphony VR* excels in usability and visual fidelity, its lack of user agency highlights the need for VR music applications to offer enhanced viewpoint control and spatial exploration. Furthermore, Wu's research on low-latency networked audio architectures [10] illustrates how future VR orchestral frameworks can expand beyond pre-recorded media toward real-time collaborative performances. Finally, Lee et al. show that translating audience feedback into

virtual avatar behaviours enhances co-presence [11], contrasting with *Symphony VR*'s solitary model to suggest the value of audience-responsive visual layers.

In summary, designing effective classical VR experiences requires harmonising high-fidelity spatial audio, meaningful viewpoint autonomy, multisensory balance, and simulated social presence.

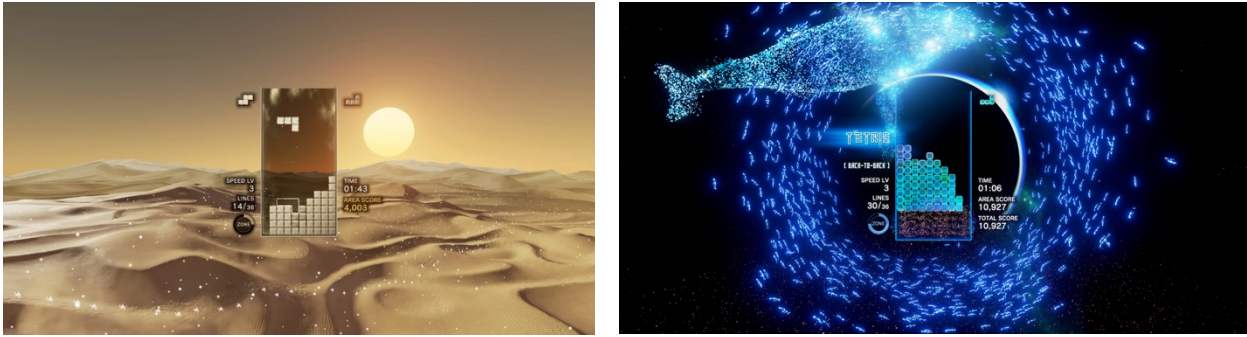


Fig. 4. *Tetris Effect* (Left: Desert theme. Middle: Ocean theme). Adapted from [12]

2.1.2. VR Presence, Immersion and Human-Computer Interaction (HCI)

Spatial presence, perceptual immersion, and psychological engagement form the foundation of user experience in VR, with *Tetris Effect* [12] serving as an exemplary comparative baseline for transforming low-agency interactions into artistic, emotionally resonant experiences through multisensory alignment. Katona defines presence as the feeling of "being there," arising from perceptual coherence and attentional focus [13]. Empirical spatial perception research further demonstrates that stereoscopic 3D depth and spatialised environments significantly improve perceptual speed, visual working memory, and cognitive engagement compared to 2D displays [14]. *Tetris Effect* leverages these mechanisms by combining stereoscopic depth, music-reactive animations, and spatialised audio to sustain spatial presence while minimising cognitive friction.

Narrative framing and environmental design significantly influence immersion and user absorption. Research on VR storytelling indicates that even minimal narrative scaffolding effectively structures user attention [15], mirrored in *Tetris Effect* by thematic "journeys" that guide mood without complex branching choices. Cultural heritage and VR tourism studies similarly show that spatial authenticity and clear environmental cues drive user engagement and enjoyment [16], [17], a principle *Tetris Effect* applies by surrounding players with responsive

visual landscapes. Furthermore, flow state research highlights that smooth perceptual feedback and rhythmic sensory loops cultivate deep psychological absorption [18], mirroring how *Tetris Effect* transforms repetitive mechanics into audio-visual flow states.

HCI considerations in stationary VR centre on comfort and cognitive load management. Cybersickness studies

demonstrate that visual anchoring and fixed reference points mitigate motion discomfort [19], [20]; *Tetris Effect* implements this by maintaining a static player viewpoint amidst highly dynamic background effects. Because excessive scene complexity increases cognitive strain and discomfort [21], *Tetris Effect* structures visual intensity rhythmically alongside musical tracks. Finally, passive VR interaction research emphasises minimising cognitive friction via subtle interface cues, such as gaze-assisted transitions, to maintain user engagement without requiring demanding agency [22].

Ultimately, *Tetris Effect* demonstrates that tight sensory alignment, stable visual anchoring, and controlled scene complexity can elevate passive VR environments into deeply immersive experiences.

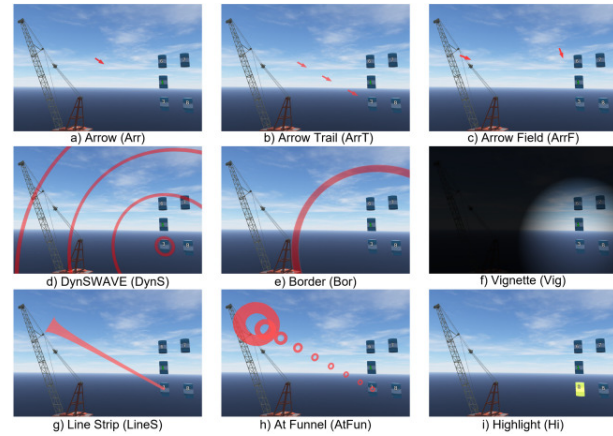


Fig. 5. Demo of visual cues. Adapted from [23]

2.1.3. Immersive Project/Team Showcases and University Environments

This review would ideally focus on immersive project/team VR showcase platforms within university environments, which aim to showcase different university groups in an engaging way. However, minimal projects of this type exist and are publicly available, so this review instead focuses on these in isolation. In other words, it focuses on immersive project showcases like virtual art exhibitions, and projects in universities like virtual campus tours.

Immersive project showcases and virtual university environments enable institutions to present achievements, academic programs, and student life within interactive 3D spaces, drawing design patterns from platforms like *Artsteps* [24] and the *UNSW 360° virtual campus tour* [25]. Decentralised curation tools like *Artsteps* empower users to construct custom virtual exhibition

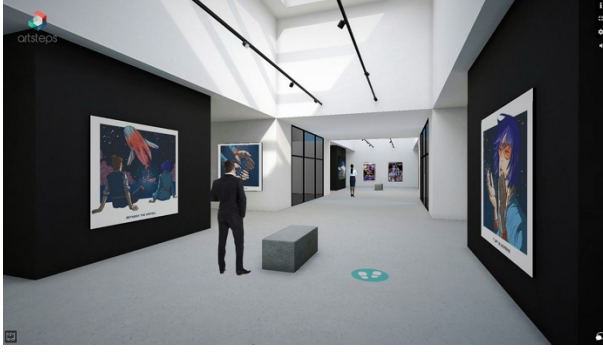


Fig. 6. Artsteps VR exhibition. Adapted from [24]



Fig. 7. UNSW 360° campus tour. Adapted from [25]

rooms, whereas 360° tour frameworks like UNSW utilise panoramic media nodes to support broad public outreach, though often remaining constrained by passive navigation models.

Recent empirical research validates the efficacy of immersive academic showcase frameworks. Almodiel et al. evaluate user acceptance in VR campus environments, reporting high perceived usefulness and ease-of-use when navigation remains intuitive and media placement is spatially coherent [26]. García et al. demonstrate that incorporating explorable 3D environments with interactive information points significantly improves user engagement compared to static media galleries [27]. To enhance navigation and user support, Wang et al. introduce *VirtuWander*, an LLM-driven virtual guide that utilises spatial cues, gestures, and spoken narration to dynamically guide visitors through complex virtual exhibits [28].

From an educational design perspective, Fernandes et al. synthesise immersive learning frameworks to highlight key inter action patterns that drive user engagement and knowledge retention [29]. On a technical level, Campos et al. demonstrate that spatial mapping tools like *ORB-SLAM3* enable precise 3D indoor reconstructions, providing a scalable foundation for modelling physical campus buildings in VR [30]. However, systematic reviews of XR in higher

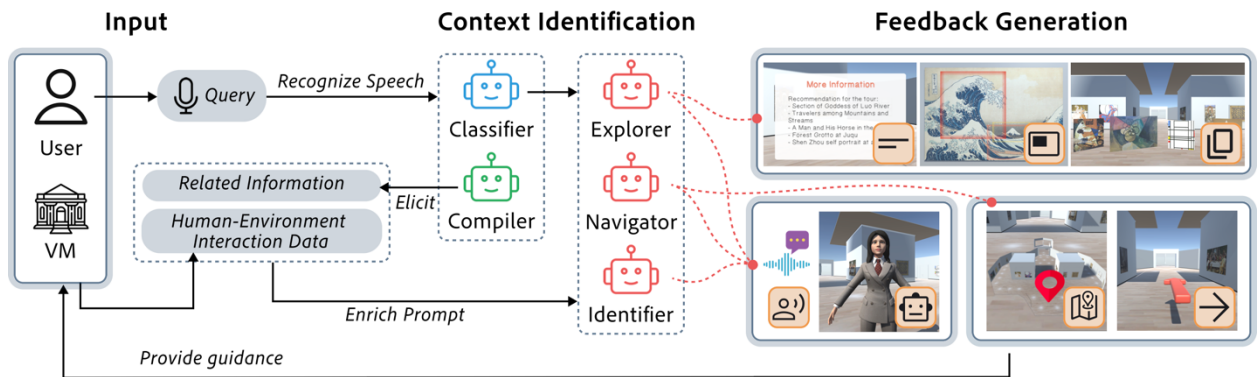


Fig. 8. VirtuWander tour guidance system with two-stage framework. Adapted from [28]

education reveal that despite widespread adoption, institutional platforms continue to face challenges regarding standardised interaction frameworks and equitable access [31].

Overall, effective university VR showcases require integrating intuitive spatial navigation, intelligent guidance, and accurate 3D architectural modelling within standardised, accessible frameworks.

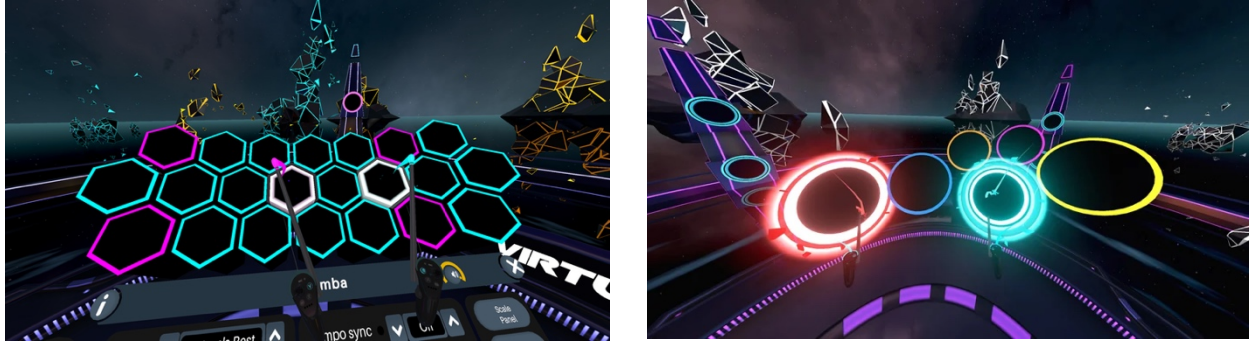


Fig. 9. *Virtuoso VR* with baton ray casting (Left: virtual marimba. Right: virtual drum kit). Adapted from [32]

2.1.4. Content Authoring and Ray Casting in VR

Ray casting and spatial authoring tools form the core interaction paradigm for immersive creative environments, enabling precise selection, manipulation, and performance in 3D space as benchmarked by creative VR applications like *Virtuoso VR* [32]. In *Virtuoso VR*, ray casting serves as the primary HCI mechanism for positioning virtual instruments and manipulating spatial controls. Research by Chen et al. demonstrates that hybrid gaze–ray selection techniques significantly increase targeting speed and accuracy over manual pointing alone, offering valuable performance enhancements for spatial audio tools and menu navigation [33].

Recent HCI studies provide deeper insight into distant target manipulation and interface ergonomics. Kim et al. evaluate *TouchView* (see Fig. 10), showing that mid-air touch metaphors improve selection precision for compact UI elements, directly addressing the precision challenges observed when adjusting small synthesiser knobs via ray casting in *Virtuoso VR* [34]. Chen et al. further highlight ray casting as an essential low-friction mechanism

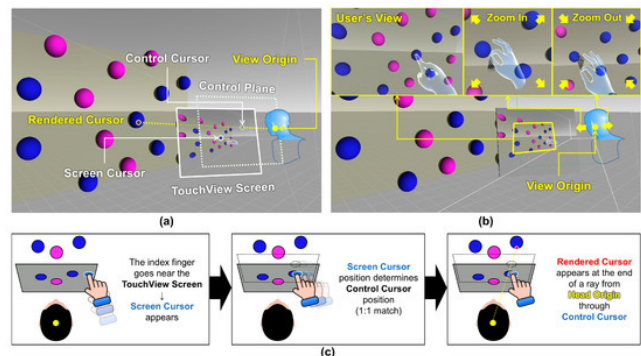


Fig 10. *TouchView* concept. Adapted from [34]

that reduces physical arm fatigue during extended spatial interaction sessions [35].

In VR content authoring, Zellerbach et al. propose a conceptual framework for mixed-reality instrument design that emphasises embodied interaction, adaptability, and live performance affordances—principles directly embodied by *Virtuoso VR*’s spatial instruments [36]. For asset manipulation, González et al. demonstrate techniques for editing interactive object behaviours in 3D space, parallel to configuring instrument layouts in virtual studios [37]. Furthermore, Benbelkheir et al. show that direct-manipulation posing tools streamline complex spatial workflows [38], while Pesek et al. prove that VR-native authoring enhances rhythmic perception and expressive timing [39].

By combining ergonomic ray casting, hybrid gaze pointing, and spatial authoring frameworks, precise, low-fatigue interaction can be delivered properly and intuitively within creative VR environments.

2.1.5. Unity Timelines, Gameplay Loops and Narrative Orchestration

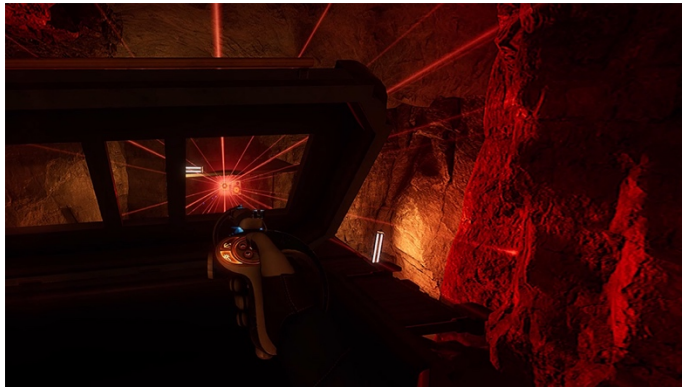


Fig 11. *Red Matter 2* stealth section. Adapted from [40]

Managing narrative pacing, interactive sequencing, and state transitions in game engines requires aligning high-level event orchestration with underlying mechanical game loops, a pipeline exemplified by narrative VR titles like *Red Matter 2* [40]. In Unity, this synchronisation is managed via the

Playable Director and Timeline systems, which orchestrate time-aligned tracks across skeletal animations, audio assets, and visual effects to maintain narrative cohesion without sacrificing performance or VR immersion.

A game's structural architecture relies on balancing player agency with deterministic progression. Chiapello formalises this through the equilibrium gameplay loop, illustrating how micro-level mechanical interactions (e.g., environmental scanning, spatial physics puzzles) feed into macro-level progression states [41]. In *Red Matter 2*, this structure ensures that completing

localised physical puzzles triggers macro-state transitions, such as opening airlocks or revealing plot audio logs. To integrate narrative beats without breaking agency, Guardiola demonstrates how linear narrative triggers must be embedded directly within gameplay loops [42]. *Red Matter 2* utilises Playable Director and Timeline tracks to execute non-intrusive scripted events—such as gravity shifts or environmental machinery activations—directly within the player's physical space, preserving room-scale VR immersion.

On a technical level, Chen evaluates real-time event orchestration in Unity, demonstrating how Playable Director components, custom Timeline marker signals, and animation tracks synchronise multi-modal feedback loops across audio, lighting, and particle systems simultaneously [43]. *Red Matter 2* relies on this framework during major environmental events, using signal emitters to trigger synchronised voiceover, prop animations, display updates, and atmospheric VFX with zero frame desynchronisation on standalone VR hardware.

Ultimately, combining Unity's Playable Director and Timeline systems with structured gameplay loops enables developers to deliver tightly synchronised, atmospheric VR experiences that harmoniously blend interactive mechanics with cinematic narrative progression.

2.1.6. Data Oriented Programming Paradigm and Technology Stack

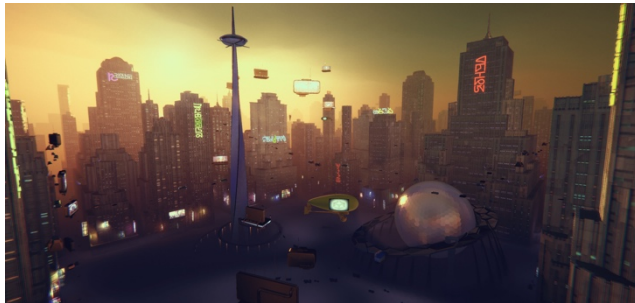


Fig 12. Megacity Metro scene. Adapted from [44]

The Data-Oriented Programming (DOD) paradigm, embodied by Unity's Data-Oriented Technology Stack (DOTS), represents an architectural shift from traditional Object-Oriented Programming (OOP) by arranging component memory contiguously to maximise CPU L1/L2 cache

locality, SIMD vectorisation, and multithreaded execution. This technical stack—encompassing the Entity Component System (ECS), C# Job System, and Burst Compiler—is benchmarked by *Megacity Metro* [44], a project demonstrating high-density VR performance by managing thousands of dynamic entities concurrently.

Engine architecture analyses emphasise the necessity of transitioning from OOP to DOD in high-entity real-time applications. Lee illustrates that traditional Unity `GameObject` architectures introduce heavy CPU bottlenecks due to pointer chasing, managed memory

allocations, and garbage collection (GC) frame spikes [45]. Unity's ECS resolves these main-thread limitations by restructuring data layouts contiguously alongside instruction execution pipelines, allowing *Megacity Metro* to stream thousands of autonomous vehicles in VR without frame drops. Performance benchmarks by Turpeinen confirm that Structure of Arrays (SoA) memory layouts drastically reduce CPU access latency [46], while Wickström et al. empirically prove that DOD architectures achieve up to 13-fold execution speedups and significantly lower cache miss rates compared to OOP models (see Fig. 13) [47].

Spatial processing and state updates similarly benefit from contiguous memory chunk allocations. Hwasser shows that executing spatial queries across flat array structures outperforms traditional class trees by eliminating heap traversal [48], while Cox et al. demonstrate that archetype-based ECS implementations maximise cache hit rates during population-wide system iterations [49]. In VR, Brookes et al. emphasise that frame rate stability is critical for user comfort, as CPU frame spikes cause visual jitter and motion sickness [50]. *Megacity Metro* prevents these latency spikes by utilising unmanaged memory blocks to eliminate GC pauses. Finally, Hansen and Öhrström show that separating logic into Systems and data into unmanaged Component arrays enables direct memory serialisation for low-latency networking [51].

In summary, *Megacity Metro* validates that Data-Oriented Design principles are essential for eliminating GC overhead and sustaining massive real-time entity counts within strict VR performance budgets.

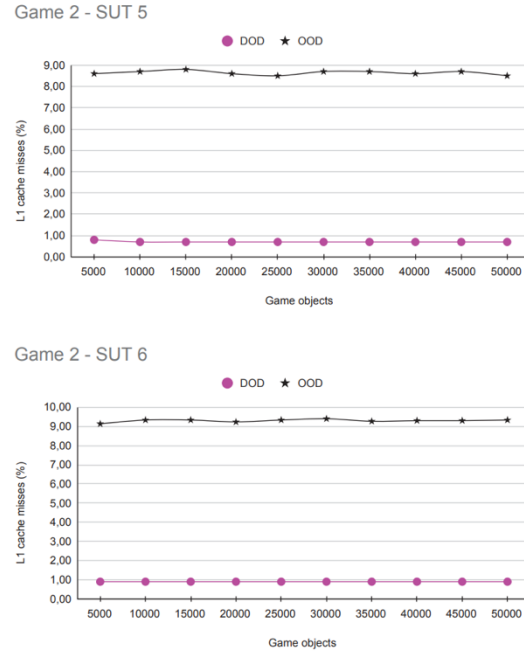


Fig 13. L1 Cache Miss results from [47]

2.1.7. Audio Spectrum Data and Audio Visualisers in Unity

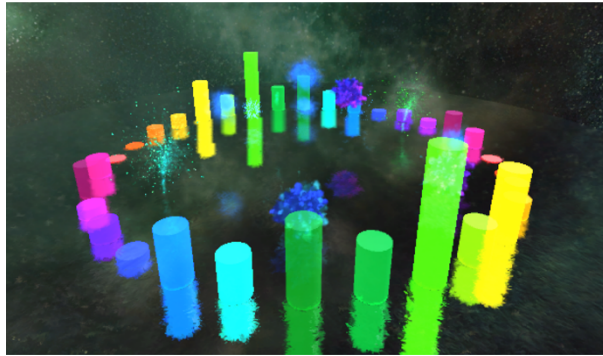


Fig 14. Celia96 music visualiser. Adapted from [52]

Real-time audio visualisation converts sound into dynamic graphical representations via digital signal processing, spectral analysis, and graphics parameter mapping, as benchmarked by open-source implementations like the *Celia96 Music Visualiser* [52]. In Unity, this pipeline extracts time-domain audio samples via Fast Fourier Transform (FFT)

algorithms, isolates frequency bands, and maps amplitude values to 3D geometry scales, particle system properties, and shader uniforms.

The core visualisation pipeline relies on converting PCM waveform data into frequency spectrum arrays. Graf, Opara, and Barthet establish a framework mapping FFT frequency bins directly to visual attributes such as mesh scale and shader parameters (see Fig. 15) [53], which operates via functions like `AudioSource.GetSpectrumData()`. To prevent spectral leakage across adjacent musical frequencies, Paisa, Henriksen, and Serafin demonstrate the necessity of applying windowing functions (e.g., Blackman-Harris or Hann) prior to FFT execution, an optimisation that forms the operational foundation of the *Celia96 Music Visualiser* [54].



Fig 15. Dynamic audio visualisation generated by combining several AV mappings. Adapted from [53]

Managing visual stability and event triggers requires temporal data processing. Klooster and Collins demonstrate that applying temporal smoothing filters—such as exponential decay or moving averages (`Mathf.Lerp`)—eliminates high-frequency visual jitter while maintaining rhythmic synchronisation, matching the buffer decay logic in the *Celia96 Music Visualiser* [55]. To expand beyond continuous amplitude scaling, Moraru and Cojocaru evaluate beat detection algorithms that monitor spectral flux across low-frequency bands to identify onset thresholds for discrete event triggers [56]. Furthermore, Kim demonstrates that passing spectrum arrays directly to GPU material properties or compute buffers avoids CPU

main-thread bottlenecks, enabling high-framerate rendering of dense reactive graphics [57]. Finally, Dimitrios shows that incorporating spatial acoustic absorption and room reverberation into audio analysis enables 3D visualisers to react dynamically to ambient environmental sound [58].

In conclusion, combining windowed FFT extraction, temporal smoothing filters, GPU buffer passing, and beat detection enables the creation of highly responsive, high-performance audio-reactive visual environments.

2.1.8. Mesh Rendering, Shader Graphs, VFX Graphs and Light Baking

High-fidelity environment rendering in VR requires balancing geometric detail, surface shading complexity, and GPU bandwidth, as demonstrated in projects like *UNSW Icon* [59] and *Maestro* [60]. Real-time graphics pipelines frequently struggle under the combined overhead of dense architectural geometry and dynamic lighting, necessitating the use of optimised mesh representations, pre-computed global illumination, and node-based visual effect graphs.

Optimising physical geometry represents the first major stage in environment authoring. Vincke et al. evaluate workflows for decimating dense point clouds and Building Information Modelling (BIM) datasets into lightweight polygonal meshes suitable for real-time VR rendering (see Fig. 16)—a workflow reflecting the geometric foundation of *UNSW Icon* [61]. To advance geometric efficiency, Z. Zheng et al. introduce a hybrid framework combining rasterisation with neural rendering, passing simplified meshes through neural feature layers to reconstruct detailed surfaces without dense geometry [62]; adopting this hybrid model allows *Maestro* to stream complex virtual environments across standalone headsets.

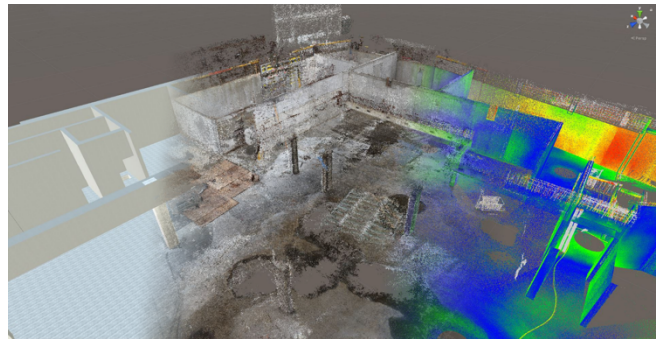


Fig 16. Photogrammetry point cloud and deviation analysis using BIM model. Adapted from [61]

Lighting optimisation and static light baking are equally critical for preserving GPU rendering headroom. Kuang et al. show that combining mesh Level of Detail (LOD) management with static light baking significantly reduces draw calls and dynamic lighting

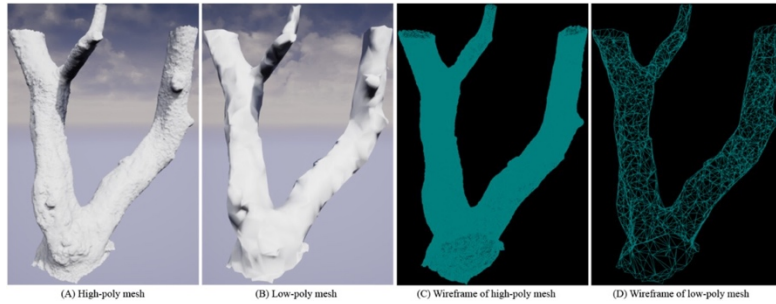


Fig 17. High and low poly mesh of tree model. Adapted from [64]

calculations in VR [63]. Similarly, C. Zheng et al. detail how texture baking bakes high-poly visual details, self-shadowing, and ambient occlusion directly into surface texture maps (see Fig. 17), allowing complex environmental

props to render efficiently on low-poly meshes [64]. In *UNSW Icon* and *Maestro*, pre-baking global illumination frees vital GPU headroom for dynamic scripts, avatar tracking, and spatial UI interactions. For standalone VR, Tanskanen proves that completely eliminating real-time lights in favour of 100% baked lightmaps drastically reduces overhead while preserving high visual quality [65]. Finally, Truong evaluates Unity's Shader Graph and VFX Graph, showing that node-based visual editors enable developers to author dynamic surface interactions and particle effects without manual shader coding—a technique *Maestro* leverages to render interactive, state-driven visual feedback in real time [66].

Thus by combining decimated geometry, pre-baked global illumination, and node-based shader graphs, VR environments can achieve high visual fidelity within strict real-time performance limits.

2.1.9. Performance Optimisation Techniques in Unity

Sustaining performance in real-time game engines requires balancing CPU execution, GPU rendering bandwidth, and memory allocation overhead, particularly when managing dense interactive scenes with high entity counts as seen in large-scale strategy titles like *Total War: Attila* (hereon *Total War*) [67]. Unoptimised rendering pipelines and memory thrashing rapidly saturate hardware limits, leading to severe frame drops that degrade user experience.



Fig 18. Level of Detail used in *Total War: Attila*. Adapted from [67]

CPU-side optimisation focuses on eliminating memory allocation spikes and streamlining spatial algorithms. Novikov highlights object pooling as a critical technique for preventing garbage collection (GC) frame spikes by reusing pre-allocated object instances during high-frequency spawning events, a method *Total War* uses to manage thousands of active projectiles [68]. Millán evaluates agent-based system optimisations, demonstrating that dynamic LOD swapping and spatial partitioning maintain playable frame rates under heavy entity workloads [69]. For spatial navigation, Kamal et al. show that optimising graph search algorithms like A* reduces node evaluations, allowing *Total War* to execute real-time pathfinding and formation movement across thousands of soldiers without starving the main thread [70].

GPU rendering efficiency requires minimising draw calls, vertex counts, and shading overhead. Le proves that texture atlasing consolidates material state changes to significantly reduce GPU draw calls, enabling *Total War* to batch soldier models via GPU instancing [71].

Scene geometry must also be culled aggressively; Singh, Sharma, and Sharma show that Occlusion Culling and LOD Group simplification eliminate hidden vertex processing passes [72]. Furthermore, combining segmented frustum culling, unlit shaders, and fake lighting authored via Shader Graph simulates dynamic visual effects without the

Scene	Average FPS (Meta Quest 3)
12xLowPoly	71.96
6xMediumPoly	48.08
3xHighPoly	10.95

Fig 19. Average FPS of different levels of detail. Adapted from [71]

computational expense of dynamic lights [72]. Profiling these optimisations empirically across FPS, batch counts, and triangle density—as demonstrated by Koulaxidis and Xinogalos—ensures balanced resource utilisation [73]. Finally, Khan, Shaikh, and Chaudhry evaluate self-adaptive framework designs that dynamically adjust operational complexity under heavy workloads, mirroring *Total War*’s runtime throttling mechanisms during peak battlefield activity [74].

In conclusion, combining object pooling, culling, texture atlasing, unlit shaders, and adaptive throttling allows real-time engines to sustain high entity counts within strict frame timing budgets.

2.2.Comparative Gap Identification

- Social & Spatial Presence in Classical Orchestral VR: Existing musical VR systems lack frameworks for translating audience presence, concert rituals, and spatialized social interaction into virtual environments.
- Autonomy vs. Comfort Balance in Immersive Musical Experiences: Guidelines remain limited on balancing interactive autonomy and motion comfort in passive/semi-passive VR without causing cognitive overload or cybersickness.
- Haptic Integration & Multisensory Audio Mapping: Standardized mappings between continuous musical features and tactile feedback are largely unexplored, limiting multisensory engagement and accessibility for Deaf/HoH users.
- Content Authoring & Raycasting Workflows for Musical VR: Current VR authoring pipelines lack unified, low-friction workflows tailored to complex, audio-reactive environments, leading to selection inaccuracy and controller fatigue.
- Unstandardized VR Showcase Frameworks for University Environments: Multi-project university VR showcases lack validated design patterns for adaptive guidance, intuitive spatial navigation, and sustained user engagement.
- Architectural Trade-offs Between DOD and Modern VR Pipelines: While Data-Oriented Design (DOTS/ECS) reduces CPU bottlenecks and cache misses, documentation on integrating ECS memory management with standard Unity rendering tools is limited.
- Zero-Real-Time-Light Optimisation vs. Dynamic Feedback: Defining visual standards for integrating dynamic, state-driven visual feedback (Shader/VFX Graphs) into fully static, lightbaked architectural scenes remains an unaddressed challenge.
- Real-Time FFT Spectral Feature Extraction vs. Rendering Bottlenecks: Research lacks comprehensive guidelines on transferring large audio spectrum arrays directly into GPU material buffers to prevent main-thread CPU bottlenecks during dense visual events.
- Deterministic Event Orchestration within Interactive Game Loops: Current literature provides limited frameworks for binding deterministic Timeline signal emitters to dynamic gameplay loops without causing visual desynchronization or breaking VR immersion.

2.3.Design Rationale

- Sensory Coherence & Multisensory Alignment: Synchronize spatial audio and visual rendering passes using windowed FFTs to drive shaders and particles directly, fostering psychological flow and visual realism.
- Controlled Visual Complexity & Comfort: Mitigate cognitive load and cybersickness by rhythmically layering visual effects alongside audio transients to maintain comfort without sacrificing visual impact.
- High-Engagement & Narrative Pacing: Structure the user experience around an equilibrium gameplay loop using light narrative scaffolding and deterministic Timeline sequences to guide attention and sustain progression.
- Precision Interaction & Low-Friction Content Authoring: Implement ergonomic raycasting and gaze-assisted selection interfaces to minimize fatigue while providing standardized authoring patterns for dynamic spatial elements.
- Multisensory Augmentation & Accessibility: Integrate frequency-mapped tactile haptics to translate musical transients into physical feedback, deepening emotional connection, spatial presence, and accessibility for Deaf/HoH users.
- Ecological Realism & University Showcase Adaptability: Optimize spatial layout and environmental fidelity to suit interactive university exhibition spaces, ensuring intuitive navigation and equitable access for first-time users.
- Performance-Driven Data & Memory Architecture: Leverage Data-Oriented Design principles (contiguously allocated memory, object pooling) to eliminate CPU spikes and garbage collection pauses, maintaining stable 90Hz–120Hz VR refresh rates.
- Hybrid Static-Baking & Dynamic Visual Pipelines: Combine pre-baked global illumination and mesh LOD simplification with Shader/VFX Graphs to minimize GPU draw calls while sustaining vibrant, reactive visual elements.
- Efficient GPU-Centric Audio Spectrum Pipeline: Apply windowing functions and temporal buffer decay to audio inputs, passing smoothed spectrum arrays directly to GPU shader parameters to bypass main-thread transformation overhead.

3. System Design and Conceptual Architecture

3.1.Design Philosophy and Experimental Nature

Unlike typical educational simulations produced by the UNSW Immersive Team—such as *Eye Sim*, which models vision impairments for optometry students—*Crescendo* is an aesthetic and experiential application. Its primary objective is to promote performances and ensembles managed by the UNSW Music Performance Unit (MPU) through captivating, audio-reactive environments rather than task-based instructional workflows.

Because stakeholder requirements evolved alongside technical constraints on standalone hardware, development followed an iterative, experimental methodology: features were prototyped, evaluated for performance and engagement, and subsequently refined or shelved. A prominent case study was the shifting wave floor (see foreground of Fig. 20), an experimental script driven by real-time distance calculations that animated floor cubes into ripple patterns during musical peaks. Empirical testing revealed severe performance degradation: frame rates dropped from 60 FPS to 48 FPS, and draw calls surged to nearly 3,000 batches (far exceeding the 100-batch budget). Consequently, the feature was shelved.

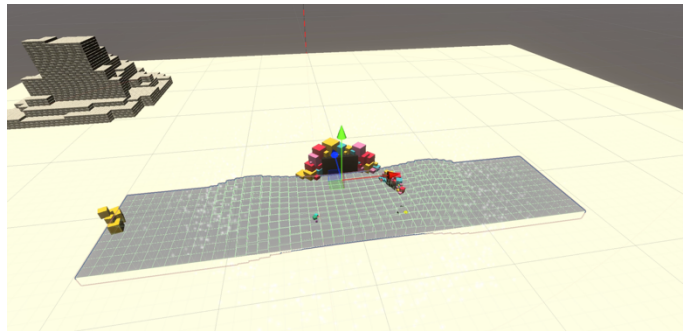


Fig 20. Shifting wave floor in development with height-map generated terrain in background.

To optimise scene generation without incurring runtime CPU overhead, the team adopted an offline baking workflow: complex geometry was procedurally generated via scripts during editing, saved directly as static prefabs, and the generating scripts were subsequently disabled. For instance, 2D heightmaps were transformed into 3D geometric structures (see background of Fig. 21), producing a blocky ceiling layout retained in the final build, while experimental background mountain terrain created with the same technique was removed.

Iterative testing yielded four primary design guidelines:

- Visual Reference Alignment: Utilising reference artwork early establishes a cohesive aesthetic target for environment design.

- Bounded Spatial Footprints: Restricting scenes to defined architectural rooms simplifies foreground/background balance and prevents empty spatial gaps.
- Single-Source Media Pipelines: Standardising on one video stream and one unified audio track drastically reduces content authoring overhead compared to multi-cube video matrices.
- Shader-Driven Visuals: Replacing dynamic light sources and transform manipulations with Shader Graph logic preserves critical GPU rendering headroom.

Finally, visual interactions were designed to preserve musical integrity. Continuous, oscillating visual cues complement soft, free-flowing musical passages, whereas sharp lighting transitions and rapid movements highlight dynamic transients, allowing non-expert listeners to interpret musical structure through visual translation.

3.2. System Architecture and Workflows

Crescendo is built on the Unity game engine using C# programming language and the Android SDK, primarily targeting Meta Quest 3 headsets but compatibility with WebGL builds in the future. The graphics pipeline utilises the Vulkan renderer for high-efficiency 3D rendering, while OpenXR handles spatial audio, device tracking, and ray casting interactions.

The data architecture decouples media assets from runtime code using Unity `ScriptableObject` assets. Each song container encapsulates track metadata, composer details, credits information, audio clips, and section-specific video textures. Integrating new repertoire requires instantiating a new asset container and assigning media files without altering scene hierarchies or source scripts.

Component interactions make good use of a couple of design patterns:

- Observer Pattern: The real-time audio analyser broadcasts frequency spectrum events to listening components (e.g., particle systems and lighting scripts) without direct coupling.
- Singleton Pattern: Globally accessible spawning manager objects handle the lifecycle and instantiation of visual prefabs across scene states.

Project assets and source code are managed via a centralised GitHub repository utilising automated CI/CD pipelines for testing, build verification, and release generation, complemented by Unity Version Control for heavy binary media assets. Due to Meta Store constraints, the team plans on reserving production APK builds for exclusive use at live UNSW VR showcase events, and due to web deployment constraints, hosted WebGL builds for public web access using the DeviceRig component have been relegated to the long term roadmap.

4. Implementation and Development Challenges

Many of the implemented features and challenges came from questions that the team asked themselves during development. Questions such as “what does this feature contribute to the experience?” and “are there ways we could implement this feature better?” This section narrates the core features being developed, some iterated over and others descope, and the design changes being made along the way. Every feature brought its own challenges and methods of problem solving, not just in the technical side of implementation, but in its design and its purposeful contribution to the engagement and immersion of the scene. It will also venture into a number of performance optimisation techniques, since this was a larger initiative that took considerable time, and the experience onboarding process, since this validated one of the aims of the project and prepared for user testing. A summary of trade-offs is listed at the end.

4.1. Core Features Development (Thesis B) — Outdoor Music Venue

During Thesis B, directly after the stakeholder meeting that determined the change to the scene composition using only 1 primary video, it was decided that the team get straight into starting over with a new scene, experimenting with displaying the screen video itself and all the various features that would complement the screen video. This phase focused on experimental features rather than a polished product suitable for user testing. It resulted in a new screen material, an audio visualiser, a voxel-type material canopy with floating cubes, foreground decorations, particle effects and an overhaul of the timeline system to use new timeline marks, linked to a centralised Game Manger script component.

1. Web build, and Device Rig

This initiative was started during Thesis A, in order to gain familiarity with the project. Many other UNSW Projects feature both a web and a VR build for extended reach and accessibility, while *Crescendo* does not. Not only would web tools significantly increase reach through deployment of a web build, but give device-specific tools that will help developers debug in the Unity editor preview.

While many options for interactivity in PC or mobile device were considered, it was decided to implement a click and drag functionality. Thesis A did involve building crosshair based interaction design, but the click and drag functionality was favoured due to ease of piggybacking from other Immersive Team projects (such as the *UNSW 360° campus tour*). Web users can click and drag with their mouse to move their camera field of view (FOV) around the entire visualisation. This meant the mouse could be retained, allowing for hover and selection interactions and even mouse scroll functionality for camera zoom within the scene.

The Device Rig became a separate object with its own components to the XR Rig. These Rigs can exist coherently, and the environment determines which is used. This means that when debugging in Unity, the Device Rig can be used for mouse interactions, but once a build is imported into the VR headset, the XR Rig takes over which handles baton interactions.



Fig 21. Audio visualiser in development with cubes that scale in size based on amplitude data.

2. Audio Visualiser and Spectrum Data

The first notable Thesis B feature implemented with considerable time investment was the audio spectrogram visualiser. 128 3D objects form a circle strategically positioned to form part of the background for the outdoor music scene. As

the music progressed, these objects immediately changed heights, forming bars matching particular frequency bands and thus responding to the music. Visualisers were familiar to most users already, and created movement that make the scene feel lively and active in what would otherwise be a very static outdoor movie theatre.

In order to achieve this, the `getSpectrumData()` function was used to get raw data from the left and right audio sources, which consists of 512 different channels directly derived from the frequency domain. The 512 channels was then divided into 64 frequency bands, each averaging groups of 8 frequencies, which were then assigned to each visualiser bar to form half the circle, and then directly mirrored to the other side to complete the circle and provide symmetrical visualiser movement surrounding the main screen.

A smoothing curve was used to ensure that the decay, but the rising, of each of the visualiser bars was not sudden. This was achieved by using the Lerp function to smooth the decay, as movement could start fast and ease over time to slow down toward the end.

Note that during Thesis C, the visualiser was moved to stick to the side walls of the room instead of forming a circle, and a number of the higher frequency bands were cut from the scene in order to display fewer bars overall in the scene. It was discovered that higher frequency bands would very rarely change, since most orchestral instruments cannot even reach those frequencies, so the scene was modified to use fewer bars that omitted these higher frequencies.

3. Wobble Screen, Voxel Canopy and Floating Cubes

A new wobble screen texture was applied to a quad object to create the main screen, piggybacked off existing rising cube screen mapping techniques. The screen also had added options that made it wobble ever so subtly, giving off the effect that the projector screen was blowing in the wind.

Surrounding the screen were a range of voxel cubes to form the canopy around the screen. This was taken as inspiration from Fig. 22, an AI generated image. The cubes themselves were the first material to have a shader graph applied to them, which gave them lots of special properties such as Fresnel power and inner glow amount,



Fig 22. Inspiration reference image for canopy.

helping to create the voxel-type texture. Additionally, four different effects were created to be purely within the shader graph for transitioning colours, all of which enabled scripts with coroutines to move sliders (floating point values) for the threshold values of these effects:

- Default: colours of all materials fade to the next colour all at the same time.
- Height: colours of all materials transition to the next colour from bottom to top based on the objects' height values.
- Radial: colours transition in an outwards motion from a centralised point in space, like a ripple effect in a pond.

- Polar: colours transition in an arc motion with a centralised point in space, like a dashboard indicator moving from left to right around the circle.

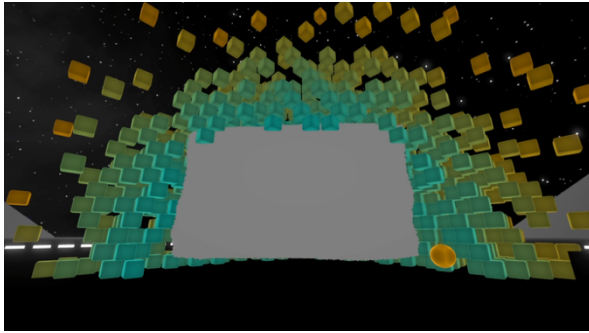


Fig 23. Radial colour changing canopy in scene.

Each transition effect changed only base colours, with their durations controlled via coroutines. During Thesis C, different experiences had different transition durations based on what was appropriate to the music (see Fig. 23).

Finally, some of the cubes floated above the rest of the canopy, to provoke a magical abstract environment. The floating was achieved by making the floating cubes move along their own sine wave function. The floating was minimal to not detract from the rest of the visualisation. Its amplitude and period of the curve, and options for rotating on its axis, could be configured further for added immersion.

4. Foreground Decorations

Cubes of different colours existed in different sizes to form the foreground elements of the scene (see Fig. 24). Each had different grayscale icons randomly assigned from an array of icons stored in Unity, which could be customised when *Crescendo* adapts to other applications. The cubes themselves had two colours, one for the fill colour and one for the detail colour (the colour of the edges of the cube and the icons themselves). These base colours could change, and switch between fill and detail colours. Then on top of that, an intensity value could control the brightness of these colours, which pulsed based on music, achieved by multiplying amplitude values by a factor to the new intensities.

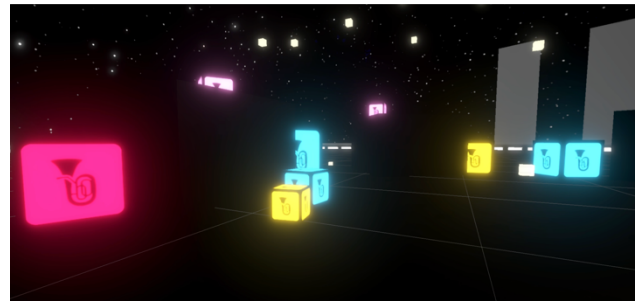


Fig 24. Decorated foreground colour cubes pulsing intensities based on amplitude data.

5. Particle Effects

Exiting Unity particle effects, and particle effects taken from the previous iteration of the project before Thesis started, proved insufficient for Thesis B due to lack of customisation and appropriateness to the new scene, plus excess lag caused by excess particles, some of which were

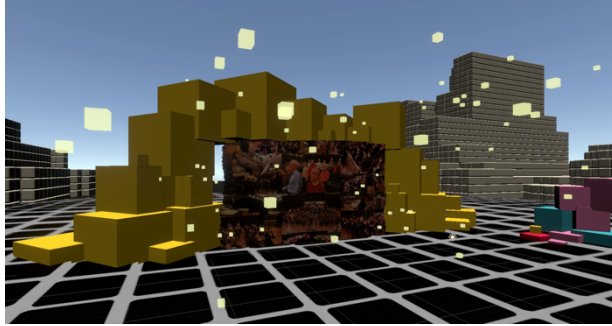


Fig 25. Particle cubes playing in development.

obstructed from camera view. Thus, time was invested in building a new custom particle system, consisting of small cubes that move upwards and disappear, transitioning their colours to match the colours of everything else in the scene (see Fig. 25). Most notably, the transitioning of the movement and the colours

follows a custom animation curve, which gets evaluated using the function `movementCurve.evaluate(blend)` in the midst of a coroutine to create custom easing. Unlike the easing used in the audio visualiser, which is fixed, the custom animation curve will allow developers to create even more interesting particle effects for longer periods of time.

6. Technical Refactors: Timeline and Game Manager

The above features were used to provide immersive visual features for viewers to observe, but a number of key fundamental technical improvements happened to the backend.

The timeline was simplified to use four key moment markers. Each moment marker can be placed in the timeline to sync with the video and the audio, cueing the above features to create dramatic

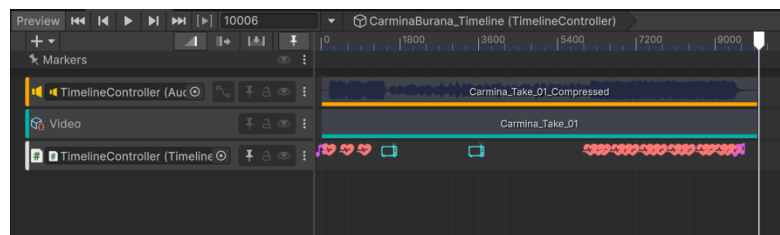


Fig 26. Timeline controller interface with moment markers

or continuous effects that happen as soon as it happens in the music. The markers include:

- *Song start*: marker to start the song, which includes setting variables up and fading the colours to black

- *Song end*: marker to end the song, which ends all existing ongoing coroutines and starts the end sequence, which stops the audio and timeline, and cues the credits in the UI
- *Big moments*: this is for dramatic effects best suited for climactic moments in the scene, such as driving rhythmic hits and peaks in musical phrasing. This really captures the essence of the intent of composers of the music in a visually appropriate way.
- *Continuous moments*: this is for effects that create continuous ongoing effects best suited for calm, soft, free-flowing moments in the music. This again captures the essence of the intent of composers of the music, in a different way to big moments that provides some ongoing engagement for viewers.

On top of this, in order to reduce authoring time, custom colour themes and baton effects were descoped. That meant a simplification of the UI sequence, with only the experience selection step granted to users. The previous iteration of the project had only 1 experience added to the system, so this step enables more experiences to be selected intuitively and with expandability in mind. Then, a dedicated Game Manager script component was created to take the input from the UI and all scriptable objects containing experience data, and assign them to the timeline to start the experience. A back button also existed to allow users to restart the UI sequence, especially useful for after an experience had ended. This Game Manager strategy proved one of the best demonstrations of Data Oriented Programming for this project.

4.2.Design Changes (between Thesis B and C)

After the Thesis B Presentation (around May 2026), the team realised “something was missing.” The spacious outdoor music venue ended up being too large of a space to have to fill, leaving many empty spaces without purpose that left users feeling lonely and uninterested. It would prove too difficult to fill that space effectively with purposeful features that integrated well together for a cohesive scene, and restrict room for expandability into theme customisation in the long term. The scene also never reached the familiarity that the team were envisioning at the start. It was still abstract in its geometric elements and did not provide any real-life comparison for users. In vague terms, it still felt like an alpha test quality build.

Thus, another scene had to be made for Thesis C. This, unlike in Thesis B, was not a complete reset of the scene. Rather, it involved a few major additions to the existing scene that

enclosed the space and removed ugly background elements, in order to create a polished scene ready for testing. Investment into performance optimisations and technical improvements helped reduce maintainability and loosen coupling.

4.3. Core Features Development (Thesis C) — Indoor Concert Theatre

The new scene for Thesis C resulted in a new enclosed grand concert hall room with pulsing neon light strips, rows of tiered seating with blocky-art-style people filling the audience and cute animations, guide lights compatible with web and XR interactions, moving light effects for big moments, continuous splat effects for softer moments, and technical refactors to the experience loop, ambient audio systems and the UI canvas.

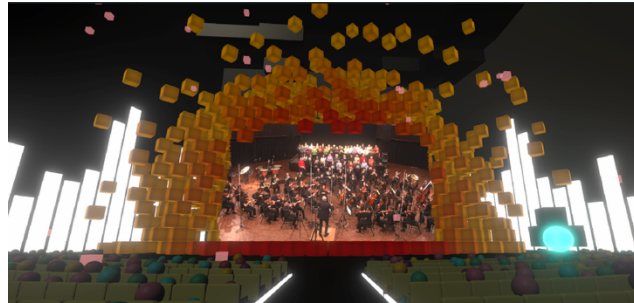


Fig 28. New indoor theatre scene in development.



Fig 27. Inspiration reference image for new scene.

1. Room Decoration

The room was changed substantially from an outdoor music venue to an indoor concert theatre (see Fig. 27). Inspiration was captured from Fig. 28, another AI generated image based off a screenshot of the original scene used in the prompt. The dark colour scheme and aesthetic was retained to focus the users primarily on the screen, and encourages lights and other bright objects to stand out more. Large walls were added and a ceiling was generated with some minor varied geometric terrain generated using procedural height-map-based generation scripts. It gives the feeling of an enclosed room to users, but this was only an illusion, as the side walls extend infinitely past the canopy without a front wall. The ground was also sloped to provide the foundation for an audience that would get added later. Visualisers bars were stuck to the side walls to replace the original circle layout, and thin horizontal beams were placed along the entire length of the side walls to break up the bars once

they rose to different heights, giving extra detail and visual interest to them. The platform was retained, to ensure that users still remain standing in the centre of the room.

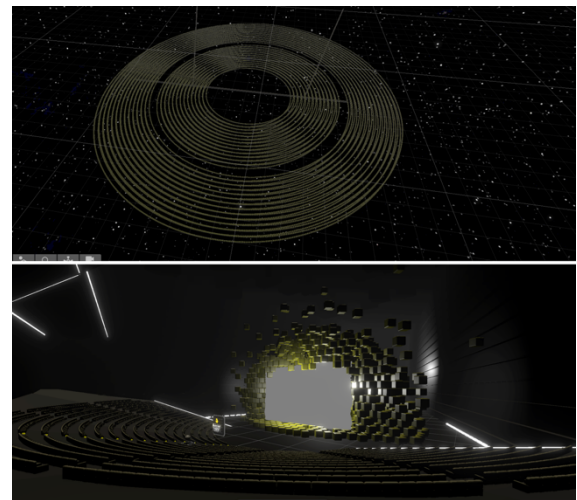
Neon light strips were added to give extra details to what would otherwise be a plain black room. These light strips were placed strategically to simulate real life guard rails and edges, turning the room into a futuristic, neon atmosphere to reinstate the abstract nature of the environment, complementing other abstract features such as the canopy. Light strips were also added above the viewer in such an arrangement that gives the illusion of an upper gallery above the user, and on the edges of the platform that the user stands on. The light strips themselves change colour matching the all other colour changes in the scene, and pulse their intensities based on amplitude of the music. They only shine during ongoing experiences, and remain black before and after experiences.

2. Audience Seating and Crowd Animations

The most notable feature added in Thesis C was the presence of an audience, which provides a sense of belonging for users within the scene and giving more familiarity to users reflecting real concert theatre experiences.

Firstly, the seats themselves consisted of backrests and armrests, constructed using simple geometry with some curved edges for refinement. This was saved as a prefab. Scripts were used to generate concentric circles for the placement of these seats (see top image in Fig. 29), which were stripped back to just a trapezium shaped area of seat rings to form the rows of seats. Another script calculated distance bands for these rows of seats and elevated them to different heights to simulate tiered seating (see bottom image in Fig. 29). The final seating arrangement was then saved as another prefab, to exist statically in the scene itself.

It is important to note that the camera still existed on its own platform away from the seats, to create a clear aisle that draws the user toward the screen. Placing a seat next to unfamiliar users



*Fig 29. Top: Seat circle rings in development.
Bottom: Tiered seating in scene.*

may tempt them to physically sit down on something that only exists in VR, so it was better to avoid this confusion by making them stand on their own special platform.

Dedicated people objects were constructed using even simpler geometry (a sphere for the head and capsules for the body and arms), saved as prefabs and then spawned on the seats via more scripts. Randomness was added to ensure not all seats were filled, and that people would appear in different sizes, even going as far as simulating children with parents, for variation and visual interest. Initially the people were also coloured randomly for more variation, however this was removed due to performance issues.

The people objects were not just existing statically in the scene. The objects themselves also highlighted themselves, by phasing to a second colour and back via coroutines and shader graph functionality. Their arms could move, either by lifting their arms in the air, or by clapping for a period of time (see Fig. 30).

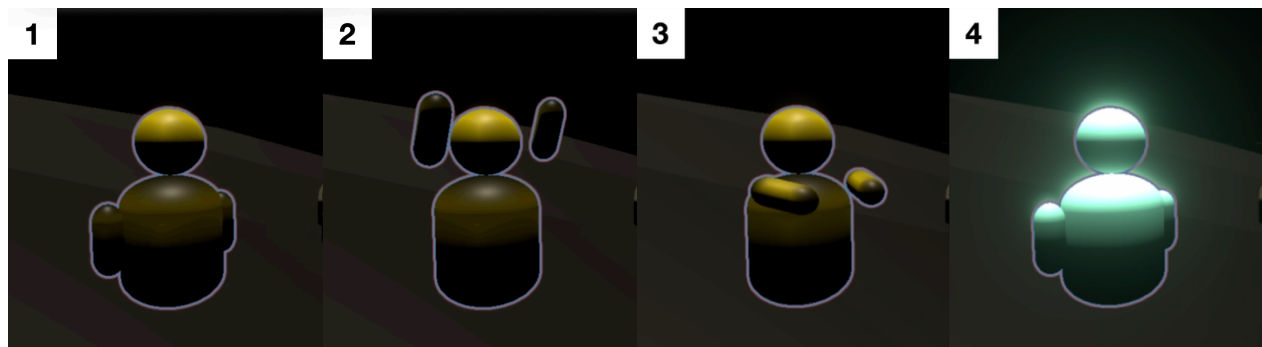


Fig 30. People objects and their movements and effects. 1: normal. 2: raised arms. 3: clapping. 4: highlighting

Each of these colour changes and people movements now had a purpose. The lifting of arms and clapping contributed to a cohesive song end experience for viewers, by having an audience applause at the end of an experience replicating in person concert experiences. When combined with ambient audio samples of an audience chatting, musicians tuning and, at the end of an experience, audience clapping and cheering, the presence of the audience became significantly more apparent, adding to the immersion and creating cheerful experiences for users both before and after actual experiences happen. Without this, before and after experience sequences would remain uninteresting and stale. The colour changes were used for the splat effect, mentioned later on in this section.

3. Interactable Guide Lights

The decorated foreground cubes were kept from the original scene from Thesis B, scaled down drastically, and placed next to seats to form guide lights along the aisles where the user would be standing. They look the exact same as before, with the same colour pulsing and iconography on them, but now have an additional feature added to them, in the form of user

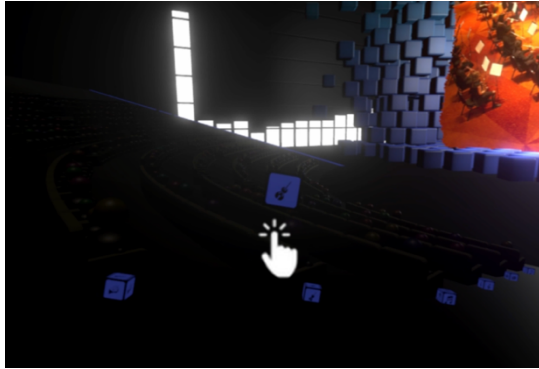


Fig 31. User in Unity picking up a guide light

interactions (see Fig. 31).

Toward the end of Thesis B, development on an interactive spherical object that could be picked up and put down via baton or mouse inputs was prototyped. Once working reliably, this behaviour was copied and added to each of the guide lights on the floor. It became a rather tedious process, because allocating script components and assigning objects to variables was almost entirely manual. Yet it served a subtle purpose in giving users “something to play with” as they discover hidden elements while the experience was running. It was always intended to be something users would discover for themselves, and developers and users alike quickly found out they could hold two cubes at the same time, which proved successful.

4. Moving Light Effects

Various light sources exist in the scene to light up the dark room, but some of these light sources moved. At dramatic big moments in the music, lights move from the canopy to the back of the room, panning across the

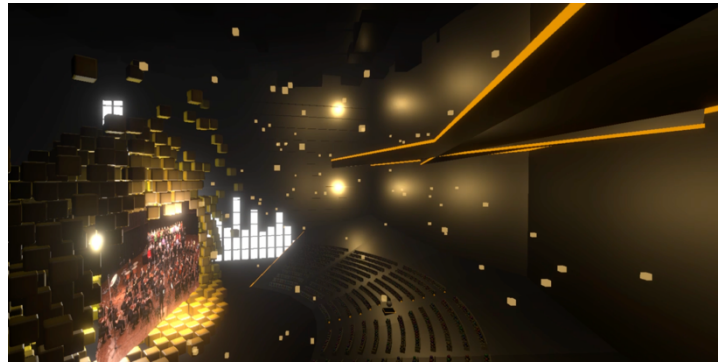


Fig 32. Moving light sources at big moments

room as the lights reflected on the surfaces of the walls and ceiling, contributing to a monumental effect that suited climactic musical moments (see Fig. 32). This was achieved through coroutines that simply moved these light source objects across the room, and then would reset them back to their initial position after the effect was finished.

This feature was eventually migrated to a fake light movement effect simulated by the shader graph, which will be discussed later in the Performance Optimisations section.

5. Splat Effect in Shader Graph

One aspect of the voxel cube material done in Thesis B but not mentioned earlier was a feature known internally as the “splat effect”. This created a highlight of a 3rd colour that propagates outwards from a different point, similar in nature to the radial transition effect but not setting the final colours of the cubes to the other colour. This forms outward circles that

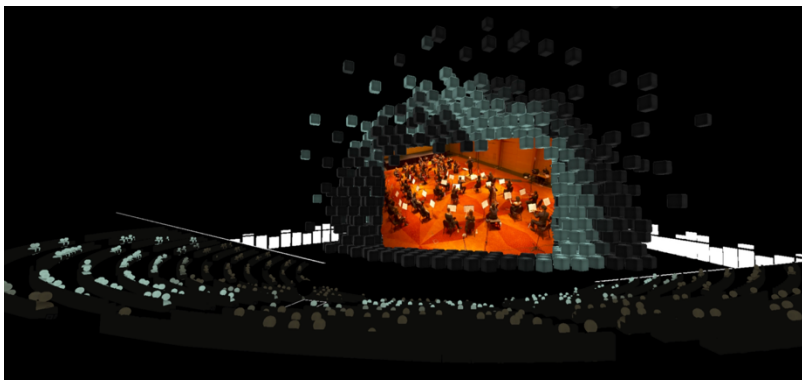


Fig 33. Splat effect in highlighting canopy cubes and audience people

propagate across the entire cube canopy, including the floating cubes. This feature was not properly implemented into the timeline during Thesis B, but when it came time to integrate the feature into the scene during Thesis C, the feature was extended to the audience as

well. Both the people in the crowd and the canopy cubes display these outward propagating circles (see Fig. 33), during strategic continuous moments in the music when enabled. Developers can tweak the speed of propagation of these circles, which proved useful for variation of timings across different experiences.

The shader graphs for the voxel cubes and the people were both refactored to include global variables that control the colour and threshold of the splat effect, so colours stand out appropriately, and the threshold can increase through a single coroutine rather than through separate coroutines in separate scripts synced to each other. In the script that enables this behaviour, the origin point for outward propagation was randomised on one axis to provide slight variation each time it happened, adding further visual interest. These global variables were essential to making this happen, but meant developers could not access these variables in the serialized field, which was the normal practice for viewing fields in the Unity editor.

6. Technical Refactors: UI Canvas and Experience Loop

Again a number of technical improvements occurred throughout development of the project in Thesis C. It made no sense during Thesis B to have the UI on its own canvas close to the user, when the screen behind could have such UI functionality displayed on it. Consequently, the UI canvas was moved to be flush with the screen. The screen itself and the UI canvas remain separate, however. This is so they can serve their respective purposes without interfering with each other. The UI canvas will show the UI and credits, with the selection menu cubes and back button now appearing directly in front of the user instead of where the screen is (simply due to baton ray cast distances), while the screen handles the assignment and playing of the video asset. It would be rather unnecessarily difficult to build a screen material that can also display UI information, thus the separated approach made development easier, loosening coupling and reducing maintenance overhead.

Now functionality for the experience loop already existed previously, but was in a broken state. After an experience was finished and after users went through the UI sequence again to start another experience, the video would fail to be reassigned, and a blank screen would appear. This would happen despite the timeline and audio still playing. After careful inspection and research, it was found that the playable director component, responsible for starting and stopping timelines, did not have capabilities for reassigning assets to fields inside of timelines, regardless of any playable graph rebuilds or bindings done. The playable director had numerous functions to help fix this issue, but none of them worked, and other users in Unity forums also had similar issues with no meaningful solutions offered.

It was decided by the team to apply a brute-force approach to solve the problem. Instead of reallocating various assets and attributes to a timeline object in the scene, the entire timeline would get removed and replaced. This required storing timeline objects as prefabs, and migrating instances of the scripts called by the timeline to the game manager to pass into whichever timeline would get played (see Fig. 34 on next page). The game manager now had more responsibilities in passing entire timelines into the scene to play, which meant each timeline's assets and attributes would be fixed to the timeline itself and never replaced (except the scripts).

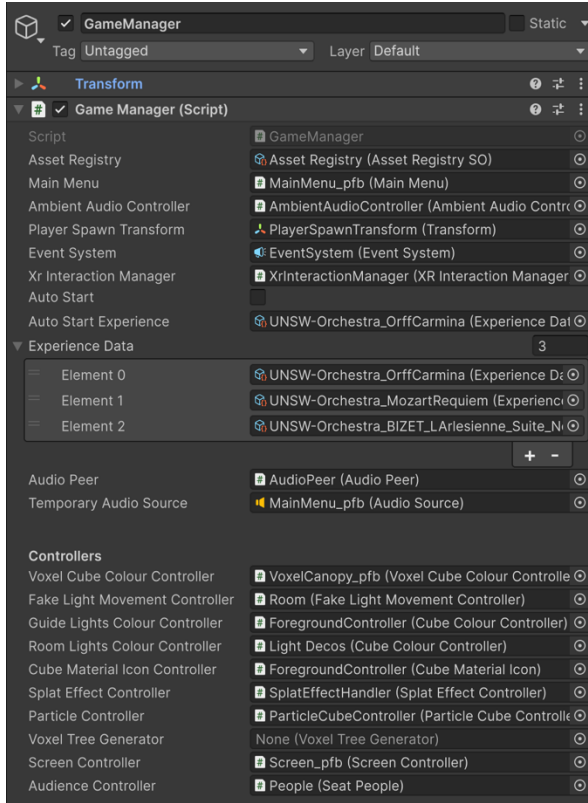


Fig 34. GameManager component updated with controller component instances to pass into Timeline Prefabs as part of experience loop

This took a considerable amount of time to implement, and was done well with maintenance overhead considerations in mind, but, thankfully, fixed the issue caused by the previous design approach.

4.4. Adding new Recordings Into the System

One of the main aims of this project was to ease the process of adding new recordings, with the aim to get recordings properly integrated into the system, with timelines and audio/video synced within a day or so. With the above features now implemented, two days were spent in July adding two more recordings into the system, resulting in four recordings currently integrated into *Crescendo* with room for many more in the future.

At the same time, the entire process was documented in the form of a procedure found in *Appendix 4.4*. This will become a useful resource for future developers to follow. The entire 9-step pipeline, from raw file input to playable immersive experience, can be configured within a day. It benefits from simplification as a result of the single video and audio file. Decoupling credits text, timeline prefabs and UI references into distinct scriptable objects ensures minimal dependencies across the scene, and since no code changes or script modifications are made in the process, open-closed software design principles are maintained. Additionally, having the four different markers gives authors precise control over lighting, particle systems and colour transitions, aligning them with specific musical phrasing to complement the orchestra score with good thought and intent.

However, the process still faces a number of flaws. By configuring timeline prefabs, playable director bindings and scriptable objects, the manual configuration introduces multiple points of

failure, so forgetting to re-bind a single component would result in broken experience loops or missing UI data. Manual placement and fine-tuning of moment markers requires listening to each piece over and over again in real time. Unlike other music editing applications, the timeline can only be inspected during active runtime play-mode (located in `DontDestroyOnLoad` hierarchy), and cannot fast forward or rewind safely, which explains the time-consuming and tedious nature of this authoring. Since the current workflow relies entirely on developers listening to identify climactic orchestral hits and soft continuous passages, the content authoring pipeline remains bottlenecked by manual labour. While automated beat detection or RMS amplitude thresholding scripts do exist, they would present many challenges when integrated with existing systems (especially when using these to pre-populate marker positions) and would require external APIs to be used, which would be out of scope of the project's requirements.

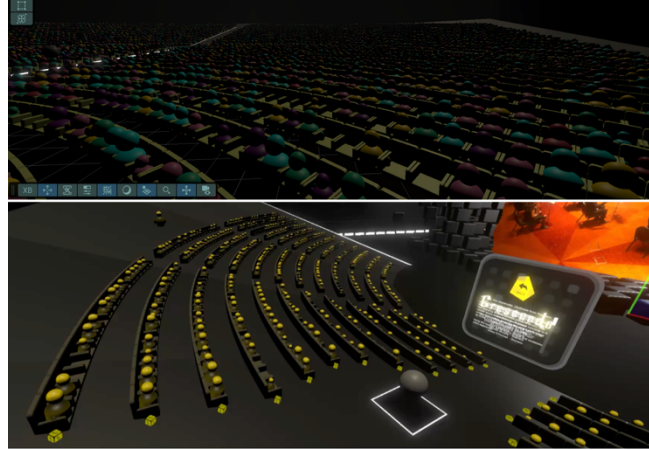
4.5. Performance Optimisations and Techniques

In around June 2026, most of the content authoring features had been finalised and the scene was almost at completion. Initial tests were conducted in VR using the OVR metrics tool to analyse performance metrics (see Section 5) while the experience was running, with FPS being most closely monitored. The first few runs through the application yielded 20 FPS, way below the desirable limit. This necessitated investigation into different methods for optimising performance, in order to increase overall FPS without compromising on the original behaviour and desired effect of the feature causing performance vulnerabilities such as lag.

1. Scene Object Reduction

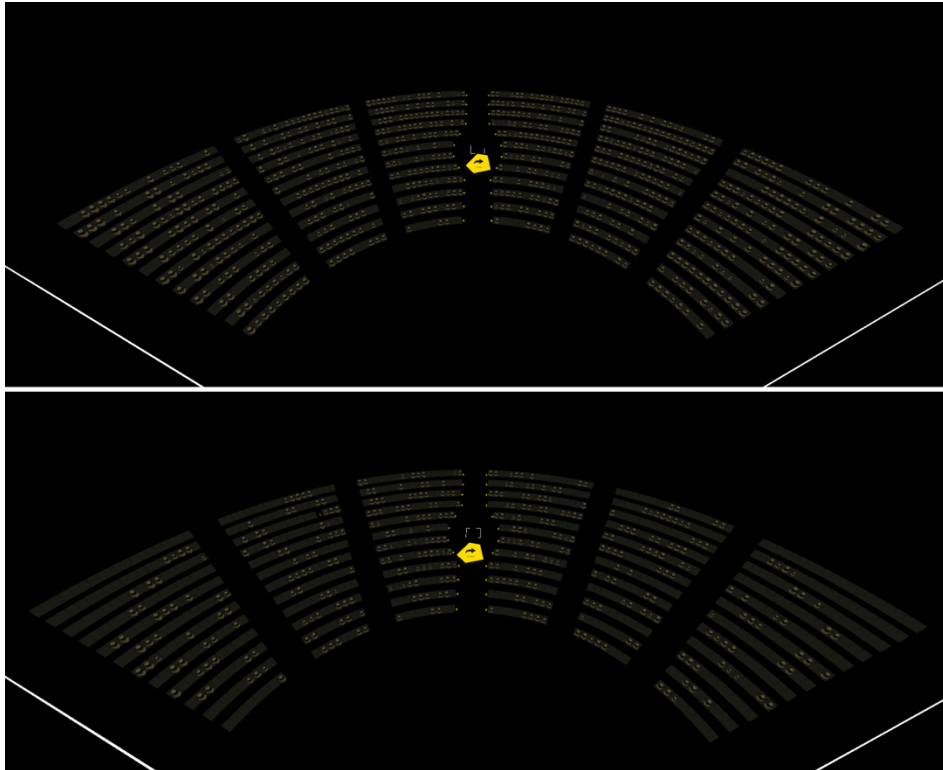
The easiest way to improve performance was to reduce the number of objects in the scene. The GPU, especially in VR headsets, struggled to render large numbers of objects (depending on their renderer, usually up to 100'000). By strategically removing unnecessary objects, the performance was improved drastically.

This was performed extensively when testing the layout and scale of the number of seats in the audience, and people objects that fill these seats. The scale was initially way too large and overwhelming for both users and the GPU, so more than half the number of seats were removed and the audience was stripped back to a more manageable state (see Fig. 35). At this point, FPS had increased significantly, but



*Fig 35. Top: way too many objects.
Bottom: much more manageable*

the team went further, manually removing people from the seats so that instead of filling 70-75% of seats with people from previous generation scripts, about 40-45% of seats would be filled with people, and a greater concentration of people would exist in seats nearer to the viewer, with seats further out being more sparse (see Fig. 36). This ensures the user feels a sense of belonging



*Fig 36. Top: seats 70-75% filled.
Bottom: seats 40-45% filled with more seats filled closer to users*

within the scene, giving the illusion that there exists a lot of people in the audience. While there may be trade-offs present in the process of removing seats and people, the team did recognise that the original seating arrangement was way too large for VR devices to handle, and the new

seating arrangement and audience people objects create presence and meaning at an appropriate scale to the room.

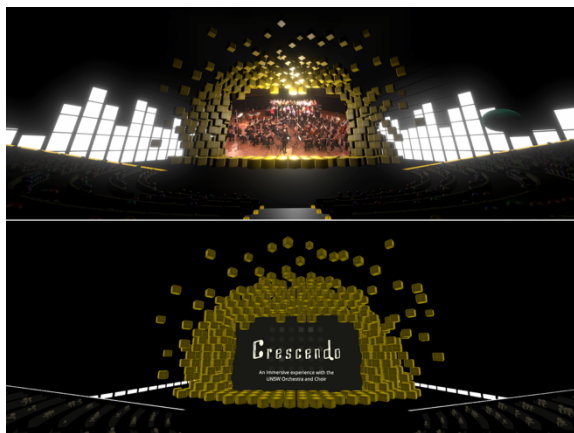
While it was difficult for the team to know which specific factors and optimisations contributed to FPS improvement, due to the nature of the testing done, it was estimated to be up to 20FPS saved through removing unnecessary objects alone.

2. Shader Optimisations: Unlit Shaders, Shadow Casting and GPU Resident Drawer

Materials in Unity can be classified as unlit or lit. The categories can be misleading here, but the difference can be seen visually in Fig. 37.

Unlit materials map a single colour or texture onto the faces of the object, and various properties such as intensity and bloom can control the brightness and emission of the colours. With bright colours, the materials can be easily seen from far away, essentially mimicking a light source as an object, especially effective in a dark room.

Lit materials do still map colours or textures onto its faces, but to a much more limited extent such that, especially in dark environments, these colours or textures cannot be seen. Instead, this type of material calculates scene lights, ambient reflections and shadows cast and received by its faces. When a light source object exists nearby, the material reflects the light on its faces to the user in ways that simulate real life lighting (like shining a light on a metallic wall). As a result, lit materials in a dark room with various light sources can combine together to create an epic neon atmosphere, and moving lights can reflect off of lit material surfaces to highlight dramatic light



*Fig 37. Top: canopy with lit shaders.
Bottom: canopy with unlit shaders.*

movements and effects to viewers, which was used at big moments in the music.

At first glance, lit materials have far more impressive capabilities that contribute to a captivating scene. However, the nature of its complex scene calculations makes lit materials susceptible to performance issues and burdens on the GPU.

One initial mitigation that helped initially was adding GPU driven rendering to each material.

This option applies the GPU resident drawer, which pre-allocates resources to reduce the

communication between CPU and GPU, thus reducing the number of lines the GPU has to draw at runtime. This option required all geometry to be static in the scene, not procedurally generated via a script at runtime. During Thesis B, GPU resident drawer options were implemented for all materials, and objects were saved as prefabs so they can exist statically within the scene. FPS improved substantially, but then after most Thesis C features were added to the build, FPS went down again.

This led the team to apply a second mitigation strategy: converting all materials to unlit, removing shadow casting from all objects, and removing light sources from the scene. The scene was adapted to use unlit materials, mostly black with some colour highlights in foreground elements and visualisers on the walls.

There also existed a number of script components attached to objects in the scene that would change individual materials' colours and properties, which was also a large contribute to lag in VR. To resolve this, those scripts were removed and objects that shared the same material had to resort to using the same colour and properties. For example, all the people in the audience had to use the same colours and intensity values mapped to the common "people" material, rather than each person having different colours. This was an acceptable change because variety still exists in the people having different sizes, positions and rates of clapping at the end of the experience.

These changes to the materials resulted in an estimated 20-30 FPS increase across both Thesis B and C, but lots of work was done to adapt the scene to better highlight the objects in the scene, to cover up the loss of the "power core" effect. This effect came from having a coloured light source behind the screen inside the canopy structure, shining light onto the faces of the cubes in interesting ways that resemble realistic reflections. With unlit cubes, the light would no longer shine reflections onto the cubes, thus making it redundant in the scene, but with the capabilities of the shader graph, the effect could be mimicked, saving on GPU calculations. The next performance optimisation discusses this effective strategy further.

3. Fake Light Movement via Shader Graph

The shader graph comes with a node-based UI that allows developers to create powerful effects that enhance material colours and properties beyond its defaults. Despite its high learning

curve, it has proven time and time again to be a huge performance saver. This is because various lighting effects and behaviours can be simulated using shader graphs, in order to reduce GPU overload at the cost of slightly diminished effects.

The best example of this mimicked behaviour in action was in the moving light effects. The original moving light sources (from Fig. 33 earlier) caused way too many issues in VR due to complex calculations required for ambient reflections and lighting changes when light sources moved across the room. Lots of time was invested in mimicking this behaviour inside of the shader graph and then applying the shader graph to the materials used on the walls of the room (see Fig. 38). Basically, a coloured circle just moves across the surface of the wall and disappears after a while, with easing added via a movement curve. This feature replaced the moving light sources in the scene, saving an estimated 10-20 FPS alone.

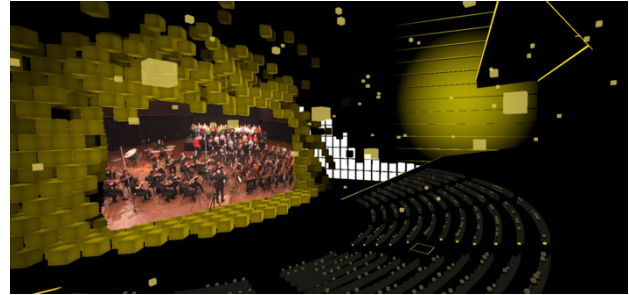


Fig 38. Fake light movement effect in shader graph applied to walls of the room.

Yet, the team have acknowledged numerous other lighting effects and places within the scene where shader graph capabilities can mimic laggy behaviour. Examples include the visualiser on the walls, guide lights on the floor, and, as mentioned previously, recreating the “power core” effect from having an origin colour light source behind the screen.

4.6.Summary of Trade-Offs in Problem Solving

- Enclosing the environment within a concert hall solved the user isolation issue caused by vast, empty spaces, but traded away the unbounded expansive scale of the original outdoor scene.
- Scrapping the shifting floor resolved severe GPU lag, camera clipping and visual overload at the expense of losing a dramatic motion effect during big moments.
- Elevating and isolating the camera on a dedicated platform prevent user confusion regarding sitting on non-existent VR seats, but ended up separating the user slightly from being embedded within the surrounding audience rows.
- Decoupling the UI Canvas from the video playback screen mesh simplified video asset management and loosened coupling. It required interaction mechanics (menu selection

cubes) to float slightly in front of the user rather than mapping directly onto screen geometry. In VR, the credits scroll bar exists too far away for users to interact with, but since the entire scroll bar is attached to the entire credits UI component, it cannot be decoupled easily.

- Removing excess seats and scaling back the people in the audience yielded an estimated 20 FPS increase, but reduced the overall visual grandness and density of the crowd.
- Removing per-person random colour scripts in favour of a single shared colour set to the material only, removed individual colour variation among audience members in order to reduce update lag.
- Transitioning all scene materials from lit to unlit eliminated light calculation overhead but lost all dynamic specular reflections and ambient wall highlights. It meant the complete removal of the "power core" lighting effect, which ended up having a significant impact on immersion, and required many changes to the scene to make up for lost immersion.
- Replacing real moving light sources with UV-mapped animated colour masks inside wall shaders recovered 10-20 FPS, but sacrificed true 3D special light emission and depth across scene geometry, which also significantly reduced overall immersion.
- The voxel canopy and crowd highlighting through the splat effect, enabled through global shader properties, allowed developers to use one coroutine to manage the entire effect across the two different material shaders, but meant these parameters could not be exposed or tweaked directly within the Unity Inspector's serialised fields.
- Replacing entire timeline prefabs within the Game Manager bypassed the asset-rebinding bug associated with the Playable Director component, to deliver a reliable experience loop, but increased overall system architecture complexity and maintenance overhead, especially evident in adding new experiences to the system.
- Relying on manual placement of moment markers ensured precise alignment with musical phrasing, but imposed tedious, time intensive authoring workflows for each new recording integrated into the system. That being said, automating this process would be an extreme challenge with only a moderate benefit to this area alone.

5. Evaluation and Feedback

In order to measure success for the project, two key areas of testing were conducted. The first in the form of feedback from actual users during testing sessions, and the second in the form of performance metrics achieved as a result of the many performance optimisation techniques done.

5.1. Usability Testing — Setup and Synthesis of Results

When testing the experience with real users, a usability test plan was used (see *Appendix 5.1*), in order to evaluate the usability of the UI selection mechanics, the engagement level of users during the experience, and the feel of interactions using the baton. Key metrics would ideally include task completion rate, ease of navigation, user engagement/boredom cues, raycast pointing accuracy and qualitative feedback on immersive audio-visually. However, due to the simple fact that the team cannot see what users are currently seeing in VR, a lot of these metrics, if feasible, would need to rely on verbal feedback from users. For example, the user would have to say “I’m seeing a bunch of cubes that change colour” in order to know they are looking at the screen canopy, or they may ask a question like “where is the back button?” to know they are struggling to navigate. These cues from encouraging users to think aloud and verbalise their thoughts will provide useful qualitative feedback to the team.

On 4th August 2026, an informal testing session was conducted to showcase the project to some UNSW undergraduate students (some being members of the orchestra itself) and gather helpful feedback. The overall feedback sentiment was positive, with comments admiring the people in the audience (especially those of smaller sizes resembling children/babies looking cute when clapping), the intuitiveness of the selection menu cubes, the vast capabilities of VR headset (i.e. the fact that all of this works in the headsets alone without any other hardware), and comments regarding the feeling of being in VR for the first time (e.g. someone said “I feel like I have lightsabers!” when referring to the batons). All the students discovered how to interact with the floor cubes themselves, some even discovering the ability to hold two at the same time, which is good interaction design. Others were amazed at the final scene composition.

However, two critical comments were made regarding potential improvements: “It would be nice if there was a way to fast forward to the end,” and, “If only I could throw the cubes.”

The first feedback comment hints at the fact that some users would likely desire better control over the playable director, by granting users the ability to pause, resume, fast forward or rewind the experiences themselves while wearing the VR headset. This would allow users to be able to restart an experience or select another one mid-way through an experience.

Regarding the second comment, it seems the current implementation of interactions feels too minimal for users. More effort could be focused into something more engaging and meaningful for interactions, such as what was mentioned in the comment: being able to throw cubes over a distance instead of having them return back to the original position.

This particular user testing session was done in the form of an informal demo, meaning the usability test plan was only followed to a small extent. With formal testing sessions booked for the future, the team can gather more meaningful results from a wider target audience.

5.2. Performance Testing — Setup, Target Metrics and Synthesis of Results

In order to know whether the experience will not lag in VR headsets, a large number of builds were created during development, uploaded to the VR headsets and then run in VR with the OVR metrics tool (see Fig. 39). The process was a lot more simple than the user test format:



Fig 39. OVR metrics tool used during live performance test session in VR build.

observations were made in VR, particularly at what happens to the different performance metrics when looking at different places within the scene, and comparing these metrics before and after the experience was started. The results for each metric and each build were documented, before discussions were conducted regarding the way forward. The team even created a

large number of builds, each with one feature disabled, in order to better determine which exact feature was causing the worst performance issues, and how best to optimise that feature.

In order to measure performance in VR headsets, various performance metrics were used. Unity contains build-in statistics that displays these performance metrics while running on computer devices, while the OVR metrics tool was used to display these statistics live in VR

while the experience was running. The team focused on three important metrics, which were the metrics used most extensively throughout the entire project, which are summarised below:

- Frames per Second (FPS): measures the number of distinct images (frames) that the VR headset displays to the user. Higher is better. 72 FPS is the required acceptance criteria for this metric set by the UNSW Immersive Team, and is the maximum value that the OVR metrics tool displays in its toolbar, which is good.
- Number of Batches: measures the number of separate instructions that the CPU sends to the GPU to draw objects. Also known as draw calls, lower is better. 250 batches is the required acceptance criteria for this metric. However this metric is harder to measure on VR because the OVR metrics tool would only show a maximum value of 100 for this metric.
- GPU Upscaling: the process of rendering frames at a lower resolution and using image and video sharpening to scale to the native resolution visible to each eye of the VR headset. The value displayed for this metric is the percentage of the full resolution (i.e. 100% means no GPU upscaling used, while 50% means resolution is scaled to 50% and then sharpened and scaled again). Higher is better. 90% is the acceptable limit for this metric, and the OVR metrics tool presents the full range up to a 100% maximum value, which is good.

Batches and GPU Upscaling metrics were used more during Thesis B development, while FPS was closely monitored and improved upon as the most important metric for success during Thesis C. This was because during Thesis C, once all features were implemented and the scene composition was nearly complete, the first test results had initially achieved the desired GPU upscaling already, and remained largely unchanged as more optimisations were made. As mentioned previously, the limitations of the OVR metrics tool meant that batches would consistently show 100 and never change, so developers had to rely on the Unity statistics for this, and it was quickly concluded that other than a few instances of over 3000 batches due to extreme numbers of objects or overcomplicated rendering materials caused by bugs that were quickly fixed. Meanwhile, FPS was constantly changing, sometimes by drastic amounts between builds, other times changing rapidly as developers looked at different places from within the scene. It proved to be the most useful metric to the team.

All performance testing results can be found in a table in *Appendix 5.2*. There were a few instances with an extremely high number of batches, but these were fixed almost immediately through GPU resident drawer options and reduction in overall objects in the scene. GPU upscaling remained largely consistent with the exception of the first build.

During Thesis B, FPS would fall substantially when the shifting wave floor was included in the scene. This was lower FPS regardless of whether the floor was moving or not (it was even worse when moving but this statistic was not recorded). However, the new scene for Thesis C dropped the FPS considerably, due to the incredibly large audience. Once that was removed, further improvements were done to optimise the scene to use unlit shaders, moving lights were simulated via shader graphs, and number of people in the audience was stripped back.



Fig 40. Testing in Unity via built-in statistics tab.

The team achieved a working build with 72 FPS everywhere in the scene except when looking at a particular location between the audience and screen, where a lot of objects existed, causing 58 FPS. From doing the trial and error method of creating multiple builds with one feature removed for each build, it was concluded that the people objects themselves (not the seats) were causing the biggest reduction in frame rate, which challenges our assumptions surrounding the people objects having been optimised enough already. Regardless, the current working build, while it remains at a usable, safe state for usability testing, unfortunately falls short of the metric standard 72 FPS all throughout the whole application.

The team currently needs time to research more ways to determine the root cause of the random lag spikes and lower FPS in specific places. From simple observations in VR, it seems the random lag spikes occur whenever a new UI cube appeared, and when the screen started playing the video, which hints at the fact that objects are still spawning into the scene or being allocated to components during runtime. There may be methods to mitigate this, but at this current point in time, it seems these features are necessary for the project to function, especially for user interactions, and cannot be mitigated at least for the short term.

6. Discussion and Critical Reflection

Development of *Crescendo* has proven to be challenging both in its design and in its technical nature. Stakeholder requirements have evolved, platform hardware limits were discovered and technical priorities kept changing. By employing an iterative, experimental approach, lots of lessons have been learnt that led to practical design guidelines for future immersive applications.

6.1. Reflections on Design Choices

Technical solutions were tailored directly to engine and hardware limits. Shader Graph logic was chosen over dynamic lighting to create smooth colour transitions and movement without overloading the GPU. When Unity's Playable Director encountered issues, C# script references were passed directly to keep the code clean and easy to maintain. A data-oriented approach using Scriptable Object assets to store track data also proved far more effective than traditional Object-Oriented programming for managing state across Timeline instances and game managers.



Fig 41. *Maestro conducting game. Adapted from [60]*

From a design standpoint, several ambitious ideas came up during stakeholder discussions—such as turning the project into a conducting rhythm game (like *Maestro* [60]), an interactive walkthrough exhibition (like *Artsteps* [24]), a 360° spatial recording platform, or an educational tool for music concepts. While compelling, these ideas fell

out of scope due to tight timelines and commitments to other projects like *UNSW Icon*. The main unresolved challenges for these shelved ideas included:

- Tracking baton and hand movements in real time to detect conducting gestures.
- Slowing down or speeding up audio and video in real time to match the player's conducting tempo.
- Designing safe virtual movement options for room-scale VR that do not rely on physical walking or other unsafe movement in the physical world.

- Capturing 3D spatial audio and converting it into baton haptics to closely recreate concert hall acoustics.

Prototyping also proved that direct testing was essential. For example, building an outdoor music venue prototype showed that open spaces felt empty and required too many 3D objects to look convincing. Moving to an indoor concert hall provided a familiar atmosphere, reduced technical demand, and gave the team a flexible base for future visual themes. Ultimately, rapid experimentation in Unity helped the team navigate shifting needs far better than a fixed, theoretical plan.

6.2. Project Limitations

Building for the standalone Meta Quest 3 revealed clear performance limits. While its mobile processor is powerful compared to older headsets, features like dynamic lighting that ran smoothly in the Unity Editor caused major frame drops on the hardware, requiring light baking instead. Additionally, Unity's built-in `AudioSource.GetSpectrumData()` method works well in VR and desktop builds, but fails completely in WebGL exports—a major limitation that still lacks a simple workaround for web deployments.

Development was also restricted to built-in Unity tools and existing internal UNSW Immersive Team components. Avoiding third-party plugins kept dependencies low, but prevented the team from using dedicated external tools (such as advanced LOD or occlusion culling packages). As a result, components like the audio spectrum analyser had to be built from scratch rather than using existing tools like the *celia96 Music Visualiser*.

Finally, project management via GitHub created significant workflow issues. While separating code into dev and main branches worked well for scripts, merge conflicts in Unity's binary files and scene assets often prevented the project from even opening in the first place. Resolving these conflicts manually was unfeasible due to the complex nature of these binary files, so the team frequently had to reset branches and re-apply changes manually.

6.3.Future Work

Future development should focus on four key areas. The first of which, continued performance optimisation, remains a current priority, required by the next UNSW event demo in order to maintain the standard of 72 FPS across the whole experience set by the Immersive Team. Although basic frustrum culling and low-poly models are already in place, further CPU and GPU profiling would be needed to spot hidden bottlenecks and apply targeted occlusion culling or mesh simplification. If these tools prove insufficient for optimising performance (e.g. lag caused by movements or audio cues), more research would be needed.

User feedback suggested adding more engaging interaction mechanics, such as interactive physics involved in throwing objects with the baton. This requires calculating throw velocity, adding colliders, scripting automatic resets and triggering fun reactions (such as orchestra members raising their arms when hit or other behaviour). This initiative would likely take a few weeks to implement.

Next, adding expanded playback control such as pause, resume, rewind and fast-forward actions, requires refactoring the main timeline structure again, and managing asynchronous coroutines. It will also involve building a pause menu with “Return to Main Menu” options, updating baton interactions and handling state resets smoothly, which would present a huge operational challenge that could take many months.

Lastly, depending on the direction set by the Immersive Team and the stakeholders for the project, the system could expand its reach to support other UNSW campus groups. This would require building a top-level “meta-loop” framework. Event staff would need to lock the application to a specific group’s content using an administrative menu (either via 3D selection cubes or even a VR keyboard to enable search functionality for lots of groups) to prevent users from switching between different groups’ experiences. This expansion would pivot *Crescendo* from a dedicated symphonic music application into a broader UNSW showcase platform, a long-term direction acknowledged by the MPU. This is by far the largest initiative that can be achieved by the current team’s skills and capabilities, taking potentially up to a year to carry out.

7. Conclusion

Through extensive content-authoring development, technical refactoring, and problem-solving, *Crescendo* has evolved into a far more innovative and immersive experience. The environment progressed from a minimalist open space with rising cubes, to an outdoor venue prototype, and finally to an indoor concert theatre offering greater realism and spatial cohesion. Audio-reactive visualisers maintain musical integrity, while dynamic lighting, colour transitions, and interactive elements boost user engagement. Additionally, refining the experience loop improved user retention, while overhauling timeline systems and asset architecture reduced content onboarding times using Data-Oriented Programming principles.

Although VR hardware limits and design concerns constrained further innovation, and the platform-wide target of 72 FPS remains unreached despite performance optimisations, user feedback confirms strong interest in further features and refinements. Whether its scope remains dedicated to MPU ensembles or expands to showcase promotional media across other UNSW groups, *Crescendo* is one step closer to fulfilling its core vision: an adaptable, accessible, and captivating immersive experience for the UNSW community.

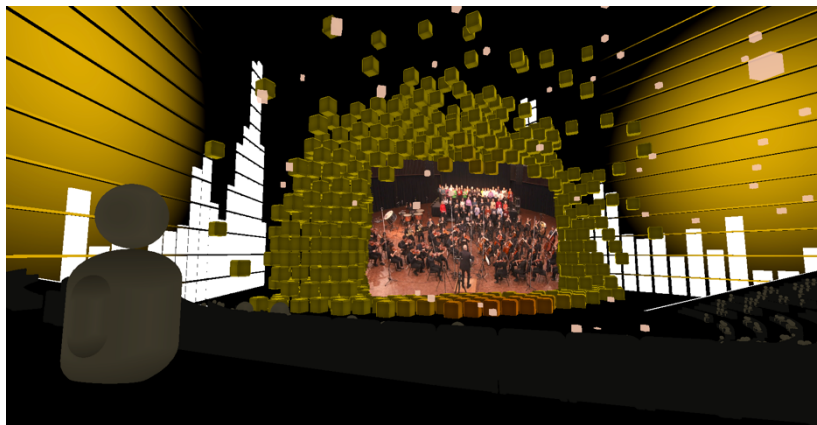


Fig 42. Final scene photo with a person object in the foreground.

Bibliography

- [1] M. Tomasetti, “Musical XR & Spatial Audio: Developing Virtual Environments for Research in Music Composition and Performance,” 13 Dec 2024. [Online]. Available: <https://iris.unitn.it/handle/11572/439195>. [Accessed 24 Nov 2025].
- [2] D. J. Aguilar, “Feeling moved in VR concerts,” 2023. [Online]. Available: <https://ir.canterbury.ac.nz/items/063d0011-bf26-4c66-9a88-a19136e9b1df>. [Accessed 24 Nov 2025].
- [3] L. Smith, “WithFeel: embodiment as a framework for accessible music composition, performance and design in virtual reality,” [Online]. Available: https://pure.ulster.ac.uk/ws/portalfiles/portal/222015167/WithFeel_embodiment_as_a_framework_for_accessible_music_composition_performance_and_design_in_virtual_reality.pdf. [Accessed 24 Nov 2025].
- [4] D. Ptman, “Sounds in Space: A portfolio of spatial music works for VR,” 2021. [Online]. Available: <https://digital.library.adelaide.edu.au/items/33d4ec17-03f5-443e-9e54-675593b2ac2f>. [Accessed 24 Nov 2025].
- [5] E. Lakka, “Immersive Sound Specification in Virtual Reality Environments,” 2025. [Online]. Available: <https://pure.southwales.ac.uk/en/studentTheses/immersive-sound-specification-in-virtual-reality-environments/>. [Accessed 24 Nov 2025].
- [6] K. E. Onderdijk, L. Bouckaert, E. V. Dyck and P.-J. Maes, “<https://link.springer.com/article/10.1007/s10055-023-00814-y>,” 5 Jun 2023. [Online]. Available: <https://link.springer.com/article/10.1007/s10055-023-00814-y>. [Accessed 17 Nov 2025].
- [7]* VR Reviews, “Orchestra VR,” 7 Oct 2016. [Online]. Available: <https://v-r.reviews/music/orchestra-vr/>. [Accessed 17 Nov 2025].
- [8] M. Lee, G. Bruder and G. F. Welch, “Exploring the effect of vibrotactile feedback through the floor on social presence in an immersive virtual environment,” 6 Apr 2017. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7892237>. [Accessed 26 Nov 2025].

- [9] J. Chen, Y. J. Lee, X. Lehto and Y. Zhu, “Exploring experience design features in the metaverse concerts: a mixed-method approach,” 18 Jun 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s40558-025-00323-5>. [Accessed 17 Nov 2025].
- [10] J. C. Wu, “Empowering Musicians: Innovating Virtual Ensemble Concert Music with Networked Audio Technology,” 14 Mar 2025. [Online]. Available: <https://www.mdpi.com/2813-2084/4/1/9>. [Accessed 17 Nov 2025].
- [11] S. Lee, Y. Cho and J. Lee, “Concert Interaction Translation: Augmenting VR Live Concert Experience using Chat-Driven Artificial Collective Reactions,” 17 Mar 2025. [Online]. Available: <https://arxiv.org/abs/2503.13121>. [Accessed 17 Nov 2025].
- [12] S. Byford, “*Tetris Effect* is out today on the Oculus Quest,” The Verge, 25 May 2020.
* [Online]. Available: <https://www.theverge.com/2020/5/14/21258435/tetris-effect-oculus-quest-release-review>. [Accessed 17 Nov 2025].
- [13] J. Katona, “A Review of Human–Computer Interaction and Virtual Reality Research Fields in Cognitive InfoCommunications,” 16 Mar 2021. [Online]. Available: <https://www.mdpi.com/2076-3417/11/6/2646>. [Accessed 21 Nov 2025].
- [14] Y. Zhao, L. Zhang, and X. Chen, "Playing Tetris in virtual reality Environment: Spatial perception, perceptual speed and visual working memory," Computers in Human Behavior Reports, vol. 18, p. 100524, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2451958825002647>
- [15] S. Yang and W. Zhang, “Presence and Flow in the Context of Virtual Reality Storytelling: What Influences Enjoyment in Virtual Environments?,” 10 Feb 2022. [Online]. Available: <https://journals.sagepub.com/doi/10.1089/cyber.2021.0037>. [Accessed 21 Nov 2025].
- [16] P. Ariya, N. Wongwan, P. Worragin, K. Intawong and K. Puritat, “Immersive realities in museums: evaluating the impact of VR, VR360, and MR on visitor presence, engagement and motivation,” 22 Jul 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s10055-025-01201-5>. [Accessed 21 Nov 2025].

- [17] S. H. Nguyen, “Leveraging Virtual Reality Experiences to Shape Tourists’ Behavioral Intentions: The Mediating Roles of Enjoyment and Immersion,” 14 Apr 2025. [Online]. Available: <https://www.mdpi.com/2673-5636/6/2/24>. [Accessed 17 Nov 2025].
- [18] Y.-C. Chang and C.-C. Yang, “Examining the Mechanisms of Virtual Reality Tourism’s Impact on the Mental Well-Being of Long-Term Care Facility Residents: Perspectives on Presence and Flow,” 6 Sep 2024. [Online]. Available: <https://www.mdpi.com/2076-328X/14/9/781>. [Accessed 17 Nov 2025].
- [19] A. Makani, R. Saryazdi, S. Givetash and B. Keshavarz, “The presence of an avatar can reduce cybersickness in Virtual Reality,” 30 Oct 2024. [Online]. Available: <http://link.springer.com/article/10.1007/s10055-024-01057-1>. [Accessed 17 Nov 2025].
- [20] S. H. Yip, A. K. T. Ng, H. Y. K. Lau and J. A. Saunders, “Preregistered test of whether a virtual nose reduces cybersickness,” 29 Oct 2024. [Online]. Available: <https://link.springer.com/article/10.1186/s41235-024-00593-3>. [Accessed 21 Nov 2025].
- [21] M. Sanaei, S. B. Gilbert, N. Javadpour, H. Sabouni, M. C. Dorneich and J. W. Kelly, “The Correlations of Scene Complexity, Workload, Presence, and Cybersickness in a Task-Based VR Game,” 27 Mar 2024. [Online]. Available: <https://arxiv.org/abs/2403.19019>. [Accessed 17 Nov 2025].
- [22] H. Cao, F. G. Rivera, H. Söderlund, C. Berlin, J. Stahre and B. Johansson, “Human-centered design of VR interface features to support mental workload and spatial cognition during collaboration tasks in manufacturing,” 20 Jun 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s10111-025-00809-6>. [Accessed 21 Nov 2025].
- [23] J. W. Woodworth and C. W. Borst, “Visual cues in VR for guiding attention vs. restoring attention after a short distraction,” Feb 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/abs/pii/S0097849323003023>. [Accessed 26 Nov 2025].
- [24] *Artsteps*, “*Artsteps* — Make your own Virtual Exhibitions,” Medium, 14 Nov 2020.
 * [Online]. Available: <https://blog.Artsteps.com/Artsteps-make-your-own-virtual-exhibitions-1cbdc3a36146>. [Accessed 21 Nov 2025].
- [25] Paper Moose, “UNSW Virtual Tour,” 2025. [Online]. Available:
 * <https://papermoose.com/our-work/case-study/unsw-virtual-tour>. [Accessed 24 Nov 2025].

- [26] M. C. Almodiel, A. M. E. F. Cañas-Llamas, M. S. Hernandez, A. M. C. Duria and Z. E. M. Villanueva, “Expanding Campus Engagement: A Study on the User Acceptance of a Virtual Reality-Enhanced Campus Tour in an Open University,” 30 Dec 2024. [Online]. Available: <https://ijitgeb.org/ijitgeb/article/view/152>. [Accessed 21 Nov 2025].
- [27] M. B. Garcia, D. M. C. Mansul, E. B. Pempina, M. R. L. Perez and R. T. Adao, “A Playable 3D Virtual Tour for an Interactive Campus Visit Experience: Showcasing School Facilities to Attract Potential Enrollees,” 7 Jul 2023. [Online]. Available: <https://ieeexplore.ieee.org/document/10169768>. [Accessed 22 Nov 2025].
- [28] Z. Wang, L.-P. Yuan, L. Wang, B. Jiang and W. Zeng, “VirtuWander: Enhancing Multi-modal Interaction for Virtual Tour Guidance through Large Language Models,” 22 Jan 2024. [Online]. Available: <https://arxiv.org/abs/2401.11923>. [Accessed 22 Nov 2025].
- [29] F. A. Fernandes, C. S. C. Rodrigues, E. N. Teixeira and C. Werner, “Immersive Learning Frameworks: A Systematic Literature Review,” 29 Aug 2022. [Online]. Available: <https://arxiv.org/abs/2208.14179>. [Accessed 22 Nov 2025].
- [30] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel and J. D. Tardós, “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” 25 May 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9440682>. [Accessed 22 Nov 2025].
- [31] D. Burke, H. Crompton and C. Nickel, “The Use of Extended Reality (XR) in Higher Education: A Systematic Review,” 17 May 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s11528-025-01092-y>. [Accessed 22 Nov 2025].
- [32] Virtuoso, “Home,” 2020. [Online]. Available: <https://virtuoso-vr.com/>. [Accessed 17 Nov * 2025].
- [33] D. L. Chen, M. Giordano, H. Benko, T. Grossman and S. Santosa, “GazeRayCursor: Facilitating Virtual Reality Target Selection by Blending Gaze and Controller Raycasting,” 2023. [Online]. Available: https://lauradichen.github.io/pdf/vrst2023_gazeraycursor.pdf. [Accessed 22 Nov 2025].

- [34] W. Kim and S. Xiong, “TouchView: Mid-Air Touch on Zoomable 2D View for Distant Freehand Selection on a Virtual Reality User Interface,” 11 Nov 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/22/7202>. [Accessed 22 Nov 2025].
- [35] M.-X. Chen, H. Hu, R. Yao, L. Qiu and D. Li, “A Survey on the Design of Virtual Reality Interaction Interfaces,” 25 Sep 2024. [Online]. Available: <https://www.mdpi.com/1424-8220/24/19/6204>. [Accessed 22 Nov 2025].
- [36] K. C. Zellerbach and C. Roberts, “A Framework for the Design and Analysis of Mixed Reality Musical Instruments,” 16 Jun 2022. [Online]. Available: <https://nime.pubpub.org/pub/mrmi-framework/release/1>. [Accessed 22 Nov 2025].
- [37] A. N. V. González, B. Williamson and J. J. J. LaViola, “Authoring Moving Parts of Objects in AR, VR and the Desktop,” 13 Dec 2023. [Online]. Available: <https://www.mdpi.com/2414-4088/7/12/117>. [Accessed 22 Nov 2025].
- [38] Y. Benbelkheir, A. Lerga and O. Ardaiz, “A Virtual Reality Direct-Manipulation Tool for Posing and Animation of Digital Human Bodies: An Evaluation of Creativity Support,” 10 Jul 2024. [Online]. Available: <https://www.mdpi.com/2414-4088/8/7/60>. [Accessed 22 Nov 2025].
- [39] M. Pesek, N. Hirci, K. Žnideršič and M. Marolt, “Enhancing music rhythmic perception and performance with a VR game,” 25 May 2024. [Online]. Available: <https://link.springer.com/article/10.1007/s10055-024-01014-y>. [Accessed 22 Nov 2025].
- [40] C. Shugg, “Red Matter 2,” The VR Grid, 16 May 2023. [Online]. Available: * <https://www.thevrgrid.com/red-matter-2-psvr/>. [Accessed 7 Aug 2026].
- [41] L. Chiapello, “Systemic Design and Game Design: The equilibrium gameplay loop,” 23 May 2024. [Online]. Available: <https://openresearch.ocadu.ca/id/eprint/4533/>. [Accessed 5 Aug 2026].
- [42] E. Guardiola and A. Czauderna, “Teaching and learning game design with the concept of gameplay loops,” 19 Jul 2024. [Online]. Available: https://intellectdiscover.com/content/journals/10.1386/jgvw_00094_1. [Accessed 5 Aug 2026].

- [43] Z. Chen, “Integrated Immersion Design in Unity3D Games: Validating Narrative, Environmental, and Interactive Cohesion,” 2 Dec 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/11257049>. [Accessed 5 Aug 2026].
- [44] T. McKenzie, “Unity Launches Megacity Metro, A New Multiplayer Game Sample,” 80.lv, 14 Mar 2024. [Online]. Available: <https://80.lv/articles/unity-launches-megacity-metro-a-new-multiplayer-game-sample>. [Accessed 7 Aug 2026].
- [45] T. Lee, “Unity Engine Dissection: Improvement Points in Comparison between Unity Engine and Open-Source Engines,” 30 Nov 2024. [Online]. Available: <http://apjcriweb.org/content/vol11no11/33.html>. [Accessed 29 Jul 2025].
- [46] M. Turpeinen, “A Performance Comparison for 3D Crowd Rendering using an Object-Oriented system and Unity DOTS with GPU Instancing on Mobile Devices,” 2020. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1467022/FULLTEXT01.pdf>. [Accessed 29 Jul 2026].
- [47] D. Wingqvist and F. Wickström, “Performance comparison between OOD and DOD with multithreading in games,” Jun 2022. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1676868/FULLTEXT02.pdf>. [Accessed 29 Jul 2026].
- [48] V. Hwasser, “Finding the Enemy: High Performing Methods for Finding Agents in Games,” Aug 2024. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1888001/ATTACHMENT01.pdf>. [Accessed 29 Jul 2026].
- [49] L. Cox, B. Williams, J. Vickers, D. Ward and C. Headleand, “Run-time Performance Comparison of Sparse-set and Archetype Entity-Component Systems,” 20 Oct 2025. [Online]. Available: <https://eprints.staffs.ac.uk/9315/1/cgvc20251224.pdf>. [Accessed 29 Jul 2026].
- [50] M. W. M. A. M. M.-W. F. M. Jack Brookes, “Studying human behavior with virtual reality: The Unity Experiment Framework,” 22 Apr 2019. [Online]. Available: <https://link.springer.com/article/10.3758/s13428-019-01242-0>. [Accessed 29 Jul 2026].
- [51] H. Hansen and O. Öhrström, “Benchmarking and Analysis of Entity Referencing Within Open-Source Entity Component Systems,” 17 Jun 2020. [Online]. Available:

- <https://www.diva-portal.org/smash/get/diva2:1479942/FULLTEXT01.pdf>. [Accessed 29 Jul 2026].
- [52] celia96, “music-visualiser,” GitHub, 2019. [Online]. Available:
* <https://github.com/celia96/music-visualizer>. [Accessed 7 Aug 2026].
- [53] M. Graf, H. C. Opara and M. Barthet, “An Audio-Driven System For Real-Time Music Visualisation,” 18 Jun 2021. [Online]. Available: <https://arxiv.org/abs/2106.10134>. [Accessed 5 Aug 2026].
- [54] R. Paisa, S. T. Henriksen and S. Serafin, “RASTA: Real-Time Audio Spectral Toolbox for Analysis in Unity3D,” 2025. [Online]. Available: <https://vbn.aau.dk/en/publications/rasta-real-time-audio-spectral-toolbox-for-analysis-in-unity3d/>. [Accessed 5 Aug 2026].
- [55] A. v. Klooster and N. Collins, “Virtual Reality and Audiovisual Experience in the AudioVirtualizer,” 19 Mar 2021. [Online]. Available:
<https://pdfs.semanticscholar.org/8b63/6504ebb85a0e9396e7ba864e067cbfe93d8a.pdf>. [Accessed 5 Aug 2026].
- [56] V. Moraru and S. Cojocaru, “Automating Rhythm Game Level Design in Unity Through Beat Detection,” 2024. [Online]. Available: https://ibn.idsi.md/vizualizare_articol/222994. [Accessed 5 Aug 2026].
- [57] D. H. Kim, “Unity Engine as a Platform for Immersive Audio Visualization & Performance Art,” May 2026. [Online]. Available:
<https://www.proquest.com/openview/6e415382d35fb9cc64f29156ce1c082f/1>. [Accessed 5 Aug 2026].
- [58] S. Dimitrios, “Perceptual Adaptation to Room Acoustics in a Unity-Based Virtual Environment,” 21 Nov 2025. [Online]. Available:
<https://ir.lib.uth.gr/xmlui/bitstream/handle/11615/87965/33278.pdf?sequence=4>. [Accessed 5 Aug 2026].
- [59] A. Steele, “UNSW students practise high-stakes conversations with immersive AI,”
* UNSW, 13 Mar 2026. [Online]. Available:
<https://www.unsw.edu.au/newsroom/news/2026/03/unsw-students-practise-high-stakes-conversations-with-immersive-ai>. [Accessed 7 Aug 2026].

- [60] J. Thompson, “Maestro Review (PSVR2) – Were You Rushing Or Dragging?,” Finger
* Guns, 23 Jul 2025. [Online]. Available:
<https://fingerguns.net/reviews/2025/07/23/maestro-review-psvr2-were-you-rushing-or-dragging/>. [Accessed 7 Aug 2026].
- [61] S. Vincke, R. d. L. Hernandez, M. Bassier and M. Vergauwen, “IMMERSIVE
VISUALISATION OF CONSTRUCTION SITE POINT CLOUD DATA, MESHES AND
BIM MODELS IN A VR ENVIRONMENT USING A GAMING ENGINE,” 20 Sep
2019. [Online]. Available: [https://isprs-archives.copernicus.org/articles/XLII-5-
W2/77/2019/](https://isprs-archives.copernicus.org/articles/XLII-5-W2/77/2019/). [Accessed 30 Jul 2026].
- [62] Z. Zheng, C. Rao, X. Zhang, L. Lu, X. Zhang and H. Wei, “Unity-EDR: A Hybrid Neural-
Mesh Rendering System for Efficient Visual Synthesis,” 18 Jun 2026. [Online].
Available: <https://ieeexplore.ieee.org/document/11562881>. [Accessed 30 Jul 2026].
- [63] Y. Kuang and X. Bai, “The Research of Virtual Reality Scene Modeling Based on Unity
3D,” 20 Sep 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/8468687>.
[Accessed 30 Jul 2026].
- [64] C. Zhang, B. He, R. Guo and D. Ma, “When a tree model meets texture baking: an
approach for quality-preserving lightweight visualization in virtual 3D scene
construction,” 1 Mar 2023. [Online]. Available:
<https://www.tandfonline.com/doi/full/10.1080/17538947.2023.2177758>. [Accessed 30 Jul
2026].
- [65] M. Tanskanen, “Lighting a Mobile Game Scene without Real Time Lights,” May 2023.
[Online]. Available:
[https://www.theseus.fi/bitstream/handle/10024/796394/Tanskanen_Mikko.pdf?sequence=
3&isAllowed=y](https://www.theseus.fi/bitstream/handle/10024/796394/Tanskanen_Mikko.pdf?sequence=3&isAllowed=y). [Accessed 30 Jul 2026].
- [66] P. Truong, “Advanced Texturing Techniques for Realistic 3D Games,” Dec 2025.
[Online]. Available:
[https://www.theseus.fi/bitstream/handle/10024/908319/Truong_Phathat.pdf?sequence=3&is
Allowed=y](https://www.theseus.fi/bitstream/handle/10024/908319/Truong_Phathat.pdf?sequence=3&isAllowed=y). [Accessed 30 Jul 2026].

- [67] Strategyturk, “Konu Mod Performance Maker for Total War: Attila,” 5 Jan 2019.
* [Online]. Available: <https://forum.strategyturk.com/konu-mod-performance-maker>.
[Accessed 7 Aug 2026].
- [68] P. A. Novikov, “Video game optimisation,” 2023. [Online]. Available:
<https://libeldoc.bsuir.by/handle/123456789/52940>. [Accessed 30 Jul 2026].
- [69] P. Galve Millán, “Ecosystem Simulator - Optimising Performance,” 14 Jul 2022. [Online].
Available: <https://upcommons.upc.edu/entities/publication/9f961861-baa1-4f47-9cce-a688ebbabd5d>. [Accessed 30 Jul 2026].
- [70] M. Nafees, L. Manlu and M. A. Kamal, “A digital twin framework for patrol robots with real-time path planning using optimised A* algorithm with ROS-Unity integration,” Jun 2026. [Online]. Available:
<https://www.sciencedirect.com/science/article/pii/S2590123026024837>. [Accessed 30 Jul 2026].
- [71] T. M. P. Le, “Optimising 3D Textures for Mobile Games,” Mar 2026. [Online]. Available:
https://www.theseus.fi/bitstream/handle/10024/918793/Le_Phuong.pdf?sequence=2&isAllowed=y. [Accessed 30 Jul 2026].
- [72] N. P. Singh, B. Sharma and A. Sharma, “Performance Analysis and Optimization Techniques in Unity 3D,” 22 Nov 2022. [Online]. Available:
<https://ieeexplore.ieee.org/document/9952025>. [Accessed 30 Jul 2026].
- [73] G. Koulaxidis and S. Xinogalos, “Improving Mobile Game Performance with Basic Optimization Techniques in Unity,” 28 Mar 2022. [Online]. Available:
<https://www.mdpi.com/2673-3951/3/2/14>. [Accessed 30 Jul 2026].
- [74] S. Garba, R. Mohamad and N. A. Saadon, “Self-adaptive mobile web service discovery framework for Dynamic Mobile Environment,” Feb 2022. [Online]. Available:
<https://www.sciencedirect.com/science/article/abs/pii/S016412122100217X>. [Accessed 5 Aug 2026].

* = non-academic source

Appendix

Appendices are numbered based on the sections they appear in, not in numerical order.

Appendix 4.4 — Experience Onboarding Process for Developers

1. Ensure a video and audio file for your recording are ready to go and both are synced to each other. Video in mp4 format. Audio in mp3 or ogg format.
2. In Unity now, drag video into Videos folder, and audio into Audio folder
3. Create a new timeline asset (or duplicate an existing one), change its name of the asset, and open it
 - a. Inside the timeline asset, delete any audios present in the audio track, and replace it with your audio file that you uploaded earlier.
 - b. Click on the video track and replace the Video clip with your video file you uploaded earlier
4. Inside the Credits folder which is inside Scriptable Objects, create a new `SongCredits` scriptable object, change the name of the object and its data to match the credits for your new experience
5. Duplicate a Timeline Prefab in the Prefabs folder, change its name, and open its properties
 - a. Ensure the playable director and its bindings has an instance to itself. You can drag the timeline you made in step 3 into that slot, and it should update automatically
 - b. In the video player component, replace the video clip with your video file you uploaded earlier
 - c. In the audio component, replace the audio file with your audio file you uploaded earlier
6. Create a new scriptable object in Scriptable Objects folder (or duplicate an existing one), change the name of the object and its data.
 - a. Drag the video into the Videos asset
 - b. Drag your timeline Prefab from step 5 into the timeline asset
 - c. Drag your credits object from step 4 into the credits asset

- d. Add an icon from inside of `Materials` -> `Selection Cube` folder, to the `Selection Cube Text Icon` asset. This will be the icon that appears on the selection cube when selecting this experience
7. You can now run the project in Unity, select the selection cube of your new experience and everything should play. The audio plays, the video appears on the screen, and the effects happen. When the `SongEndMarker` is reached, the credits display and scroll and the audience claps.
8. Open your timeline again (in `Videos` folder, not the timeline Prefab). Spend the (tedious) amount of time listening to the experience over and over again, adding/removing/moving markers around, changing their properties. You can't see the timeline move by itself until you play the experience, at which the cursor will only move when you open `DontDestroyOnLoad` -> `GameManager` -> `Timeline Asset` while an experience is playing. Remember to save the timeline after every change. Rules to note in the process:
 - a. A `SongStartMarker` must exist at 0s
 - b. A `SongEndMarker` must exist at the end of the experience. Give about 1-2s of silence after the audio ends so the user can register the ending before the clapping starts (triggered from this marker)
 - c. Ensure they sync with the audio and colours/effects complement the music.
 - d. `BigMomentMarkers` are designed to be big hits that happen at key climactic points in the music, or for a constant rhythmic drive (e.g. when timpani hits, or at landing point in musical phrasing).
 - i. Intensity mapped to: 0 = minor, 1 = major (particles), 2 = super major (particles + fake light movement on wall).
 - ii. Voxel mode is the mode for the big canopy around the screen. Colours can transition via: `Radial` (outwards from bottom of screen), `Height` (bottom to top), `Polar` (currently not working) or `AllAtOnce` (all transition at the same time, default)
 - iii. Change duration required, the duration for each colour change

- e. `ContinuousMomentMarkers` are designed to happen continuously at soft points in the music.
 - i. Requires a colour palette to cycle through and duration of each colour change.
 - ii. You can also activate a splat effect
- 9. When you are happy with the experience, save the timeline and the project. Add, commit and push to GitHub.

Appendix 5.1 — Usability Testing Outline

Test Overview & Objectives

- Core Goal: Evaluate the intuitive usability of the UI selection mechanics, the engagement level of the visualizer/lighting effects during orchestral playback, and the feel of the baton-based interactive floor objects.
- Key Metrics: Task completion rate, ease of menu navigation, user engagement/boredom cues, raycast pointing accuracy, and qualitative feedback on immersive audio-visuals.

Participant Screening Criteria

To ensure reliable, actionable data, participants should meet the following baseline criteria:

- Demographics: Mixture of first-time VR users and experienced VR users (to test menu intuitiveness across skill levels).
- Accessibility & Comfort: Must not be highly prone to severe motion sickness or cyber-sickness; comfortable standing or sitting with a VR headset for ~5–15 minutes.
- Audio/Music Background: Open to classical or orchestral music (no specialized musical skill required).
- Exclusions: History of photo-sensitive epilepsy (due to lighting and dynamic visualizer effects).

Data Gathering & Feedback Methods

- Think-Aloud Protocol: Participants are encouraged to verbalize what they are seeing, thinking, and feeling as they interact with the menu and floor objects.
- Moderator Observation: A facilitator logs interactions, noting where the user hesitates, struggles with baton pointing/grabbing, or displays signs of disinterest/boredom.

- Post-Test Semi-Structured Interview: A structured verbal Q&A session directly after removing the headset (Step 8) to capture overall impressions and feature requests.

Test Execution Procedure (Step-by-Step Task Flow)

1. Put on the VR headset with the app already open to the first UI page, and give the user their batons.
2. Get user to observe the scene around them, then select the selection cube in front of them. Observe the pressing of baton buttons is intuitive to user who's never used VR before.
3. Get user to choose "O Fortuna" from Carmina Burana by Carl Orff, and then click the next selection cube to confirm their selection. If they need help, tell them it's on the far left.
4. Watch the entire experience, noting the visualizer, lighting, colour effects, and audience response.
5. If bored, get them to point the baton at the floor cubes and hold down to pick them up, and release to release it. Observe user's hand positioning on batons to verify "hold and release" feels ergonomic during interactions
6. At the end of the experience, get them to look at the audience applause. At this point, if the user wants to finish, they can take off the VR headset and skip to Step 8. Otherwise, continue to Step 7.
7. Get user to press the back button, go through the menu process again, and select another experience of their choosing, repeating steps 4–6. They may choose any experience they like. Some notes to say on each:
 - a. Des Ires (Requiem): us experimenting with an experience without the orchestra on a video, showing we're open with different media
 - b. O Magnum Mysterium: very soft and different experience of recording from UNSW Wind Symphony, climactic moments are more epic, different effects (e.g. splat effect)
 - c. Intermezzo (Bizet): another mixed (but largely soft, free-flowing) orchestral recording experience from UNSW Orchestra only, different effects (e.g. splat effect)

8. After that's all done, take the VR headset off and give back batons. Ask the user for their overall thoughts on the experience and any improvements they would like to propose.
9. Clean headsets/batons and give them to the next person.

Appendix 5.2 — Performance Testing Results Table

Build no.	Description of the Build	Omitted features	FPS	Batches	GPU Upscaling
<i>Thesis B — Outdoor Music Venue scene</i>					
1	All features for outdoor scene	n/a	12 looking at canopy screen, 39 looking away	4500	80-90%
2	GPU resident drawer enabled and most objects pre-existing	Shifting wave floor	53 looking at canopy screen, 68 looking away	100-250	90-99%
2a	in scene instead of generated at runtime	n/a	46 looking at canopy screen, 60 looking away	100-250	90-99%
<i>Thesis C — Indoor Concert Theatre scene</i>					
3	All features for new indoor scene, extremely large audience, lit shaders everywhere, shadows enabled, directional and moving light sources	n/a	< 20, in Unity! Not even in VR yet	5170	90-99%
4	Stripped back audience significantly with	n/a	20 (now in VR) looking at anything but the ceiling, 72 looking at ceiling only	100-250	90-99%
4a	smaller room	Moving light	28 looking at canopy/audience, FPS looking at ceiling	100-250	90-99%

		sources only	unchanged from this point onward		
4b		All light sources	32 looking at canopy/audience	100-250	90-99%
4c	All canopy cubes and people converted to unlit shaders	All light sources and people on the seats	38 looking at canopy/audience	100-250	90-99%
4d	All seats and visualiser cubes converted to unlit shaders	All light sources, people,	50 looking at canopy/audience	100-250	90-99%
6	All materials in scene unlit with shadows removed	Moving light sources only	50 looking at screen, 58 looking at audience	100-250	90-99%
6a	Audience people stripped back to 40-45% seats filled	Moving light sources only	52 looking at screen, 68 looking at audience	100-250	90-99%
9	Moving lights replaced with fake light effect in shader graph, thus no more light sources	n/a	68 looking at screen, 72 elsewhere (FPS looking between screen and audience not documented yet)	100-250	90-99%
9a		Visualiser	68 looking at screen, 72 elsewhere, 52 looking directly between audience	100-250	90-99%

			and screen (most features in FOV)		
9b		Visualiser and people	68 looking at screen, 72 elsewhere, 58 looking directly between audience and screen	100-250	90-99%
10	Final scene ready for testing, audience no longer changes colour via script, only via material	n/a	72 everywhere, 58 looking directly between audience and screen, sudden 50 FPS split-second spikes at random points in time	100-250	90-99%